On the Decidability of $\mathcal{ALCI}_{RCC5}$ and $\mathcal{ALCI}_{RCC8}$: Quasimodels Meet the Patchwork Property

Claude*

March 2026 (revised)

Disclaimer. This paper was authored entirely by Claude (Anthropic), an AI assistant, prompted by Michael Wessel. The results and proofs presented here have **not been peer-reviewed or verified by human domain experts**. They are published as a discussion piece for the description logic and spatial reasoning communities. The claims should not be taken as established results unless independently verified or refuted by experts in the field. We invite scrutiny, corrections, and feedback.

*Claude is an AI assistant by Anthropic. This research was prompted by Michael Wessel (miacwess@gmail.com), who introduced the $\mathcal{ALCI}_{RCC}$ family in his doctoral work at the University of Hamburg and left the decidability of $\mathcal{ALCI}_{RCC5}$ and $\mathcal{ALCI}_{RCC8}$ as open problems [12, 11].

Status of the proof (April 2026, second revision). The decidability of $\mathcal{ALCI}_{RCC5}$ and $\mathcal{ALCI}_{RCC8}$ is **not established** by this paper. The proof contains **three genuine errors**:

Error 1: The type elimination algorithm is unsound (Theorem 6.1). The concept $C_1 = \exists\mathsf{PO}.D \sqcap \exists\mathsf{DR}.(B \sqcap \forall\mathsf{PO}.\neg D) \sqcap \forall\mathsf{DR}.\neg D \sqcap \forall\mathsf{PP}.\neg D \sqcap \forall\mathsf{PPI}.\neg D$ is satisfiable (3-element model verified computationally), yet the algorithm eliminates all 128 types to zero. The root cause: condition (Q3) is *anti-monotone* in the type set—starting with all types makes (Q3) maximally restrictive, killing DN entries needed for (Q1) witnesses, causing cascade elimination of *all* types. The model-extracted types form a valid quasimodel among themselves, but the greatest-fixpoint algorithm cannot find this subset. Theorem 6.1 is **wrong**; see Remark 6.2.

Error 2: (Q3) soundness proof gap. The proof of (Q3) in the soundness direction (Section 5.1) handles the case $\tau_2 = \tau_3$ with a unique element by setting $R_{23} = \mathsf{EQ}$. But DN is defined over $N_R \setminus \{\mathsf{EQ}\}$, so $\mathsf{EQ} \notin \mathsf{DN}(\tau_2, \tau_3)$. The proof silently uses EQ in a domain that excludes it.

Error 3: Tableau soundness depends on the Henkin construction (Corollary 7.12). The tableau calculus (Section 7) terminates and is complete, but the soundness proof (Theorem 7.10) extracts a quasimodel from an open completion graph and invokes the Henkin construction (Theorem 5.1). The Henkin construction has the *extension gap* (Remark 5.3): at each step, a disjunctive constraint network must be solved, and global solvability is not guaranteed—1,911 counterexamples to the pairwise-implies-global claim were found computationally at $m = 3$. The tableau is therefore **not a proven decision procedure**.

What survives. The quasimodel extraction from a model (soundness direction) is correct for the case where all types have multiplicity ≥ 2, or when (Q3) is restricted to pairwise-distinct types. The pair-type definitions, composition table analysis, patchwork property usage, and Henkin construction are individually correct. The identification of the extension gap is accurate. The tableau's termination and completeness proofs are independent of the type-elimination errors.

What fails. The "no false negatives" claim is wrong. The algorithm can reject satisfiable concepts. The tableau cannot be shown sound. The decidability of $\mathcal{ALCI}_{RCC5}$ and $\mathcal{ALCI}_{RCC8}$ remains **genuinely open**.

Abstract

The description logics $\mathcal{ALCI}_{RCC5}$ and $\mathcal{ALCI}_{RCC8}$ extend $\mathcal{ALCI}$ with role boxes derived from the composition tables of the Region Connection Calculi RCC5 and RCC8. These logics were introduced by Wessel [12], who established PSPACE-hardness of $\mathcal{ALCI}_{RCC5}$ and EXPTIME-hardness of $\mathcal{ALCI}_{RCC8}$ but left decidability open. We present two procedures—a *type-elimination algorithm* and a *tableau calculus with blocking*—that exploit the *patchwork property* of the RCC calculi [10] and the full tractability of the RCC5 algebra [9]. **Erratum (April 2026):** Neither procedure establishes decidability. The type-elimination algorithm is **unsound**—it rejects satisfiable concepts (see Remark 6.2). The (Q3) soundness proof has a gap for singleton-multiplicity types ($\mathsf{EQ} \notin \mathsf{DN}$). The tableau calculus terminates and is complete, but its soundness proof depends on the Henkin construction, which has an unresolved extension gap (Remark 5.3). The decidability of $\mathcal{ALCI}_{RCC5}$ and $\mathcal{ALCI}_{RCC8}$ remains **genuinely open**.

Keywords: description logic, spatial reasoning, Region Connection Calculus, decidability, tableau, quasimodel, patchwork property

1 Introduction

Description logics (DLs) are a family of knowledge representation formalisms that underpin ontology languages such as OWL [1]. The *standard* decision procedures for DLs exploit the *tree model property*: every satisfiable concept has a tree-shaped model, so one can search for a finite abstraction of such a tree (a "type elimination" or "tableau" argument).

The $\mathcal{ALCI}_{RCC}$ family, introduced by Wessel [12, 11], takes a different path. Here the roles are not abstract binary relations but the *base relations* of a qualitative spatial calculus—specifically, a Region Connection Calculus (RCC) [8]. Interpretations are constrained to be complete graphs in which every pair of domain elements is related by exactly one base relation, and the composition of any two base relations along a path is required to conform to the RCC composition table.

Formally, let $\mathcal{T}$ be one of RCC1, RCC2, RCC3, RCC5, or RCC8. The logic $\mathcal{ALCI}_{RCC}[\mathcal{T}]$—or simply $\mathcal{ALCI}_{RCCk}$ for the standard RCC-k variants—is the extension of $\mathcal{ALCI}$ (the basic DL $\mathcal{ALC}$ enriched with inverse roles) whose role names are the base relations of $\mathcal{T}$, and whose interpretations are subject to the *one-cluster, JEPD, composition table, and converse constraints* (Section 2).

Wessel [12] classified most of the family:

- $\mathcal{ALCI}_{RCC1}$ is NP-complete (equivalent to the modal logic S5).
- $\mathcal{ALCI}_{RCC2}$ is decidable (reduction to a two-variable fragment).
- $\mathcal{ALCI}_{RCC3}$ is decidable (reduction to the two-variable fragment of first-order logic with equality, $\mathrm{FO}^2[=]$).
- $\mathcal{ALCI}_{RCC5}$ is PSPACE-hard; decidability was left open.
- $\mathcal{ALCI}_{RCC8}$ is EXPTIME-hard; decidability was left open.

The difficulty with $\mathcal{ALCI}_{RCC5}$ and $\mathcal{ALCI}_{RCC8}$ stems from the combination of three properties: models are complete graphs (no tree model property), infinite models are required (no finite model property), and the composition tables have non-deterministic entries (the role boxes are not "admissible" in the sense of $\mathcal{ALCRA}_{SG}$ [11]).

Contribution. We prove that concept satisfiability in $\mathcal{ALCI}_{RCC5}$ and $\mathcal{ALCI}_{RCC8}$ is decidable, with an EXPTIME upper bound. Combined with the known lower bounds, this yields EXPTIME-completeness for $\mathcal{ALCI}_{RCC8}$. For $\mathcal{ALCI}_{RCC5}$, the exact complexity remains between PSPACE and EXPTIME.

Proof techniques. We develop two complementary decision procedures. First, a *quasi-model method* (Sections 4–6) that abstracts a (possibly infinite) complete-graph model into a finite structure consisting of *types*, *pair-types*, and *triple-types*, and a type-elimination algorithm deciding quasimodel existence. Second, a *tableau calculus* (Section 7) that constructs a finite completion graph by expanding individual nodes—analogous to standard DL tableaux, but adapted to complete-graph semantics. In the tableau, each new node is assigned a role to *every* existing node, and a blocking condition based on type equality ensures termination. The key technical ingredient in both procedures is the *patchwork property* of RCC5/RCC8 [10, 9]: path-consistent atomic constraint networks are globally consistent.

Related work. Lutz and Wolter [7] studied modal logics with RCC8 topological modalities interpreted over actual topological spaces; in that setting, reasoning is generally undecidable. Lutz and Miličić [6] showed that $\mathcal{ALC}$ extended with ω-admissible concrete domains (which include RCC5 and RCC8) is decidable, but the concrete-domain approach does not allow quantification *over* spatial roles (the defining feature of $\mathcal{ALCI}_{RCC}$). Borgwardt et al. [2] further refined the complexity of concrete-domain reasoning. The $\mathcal{ALCRA}_{SG}$ fragment [11] handles *admissible* role boxes (deterministic, functional, complete, associative) via reduction to $\mathcal{ALC}$ with general TBoxes, but the RCC5/RCC8 role boxes are non-deterministic and hence fall outside $\mathcal{ALCRA}_{SG}$.

2 Preliminaries

2.1 The Description Logic $\mathcal{ALCI}_{RCC5}$

Definition 2.1 (Syntax)**.** Let N_C be a set of *concept names* and let $N_R = \{\mathsf{DR}, \mathsf{PO}, \mathsf{EQ}, \mathsf{PP}, \mathsf{PPI}\}$ be the set of *RCC5 base relations*, serving as role names. *Concepts* are built by the grammar:

$$C, D \ ::= \ A \mid \neg C \mid C \sqcap D \mid C \sqcup D \mid \exists R.C \mid \forall R.C \mid \exists R^{-}.C \mid \forall R^{-}.C$$

where $A \in N_C$ and $R \in N_R$. The *inverse* mapping on roles is $\mathsf{inv}(\mathsf{PP}) = \mathsf{PPI}$, $\mathsf{inv}(\mathsf{PPI}) = \mathsf{PP}$, $\mathsf{inv}(\mathsf{DR}) = \mathsf{DR}$, $\mathsf{inv}(\mathsf{PO}) = \mathsf{PO}$, $\mathsf{inv}(\mathsf{EQ}) = \mathsf{EQ}$.

We assume concepts are in *negation normal form* (NNF): negation is pushed to concept names. Every $\mathcal{ALCI}_{RCC5}$ concept can be converted to NNF in linear time. In NNF, inverse roles R^{-} can be absorbed: $\exists \mathsf{PP}^{-}.C = \exists \mathsf{PPI}.C$, etc. We therefore write role terms simply as elements of N_R.

Definition 2.2 (Semantics)**.** An $\mathcal{ALCI}_{RCC5}$ *interpretation* is a structure $\mathcal{I} = (\Delta^{\mathcal{I}}, \cdot^{\mathcal{I}})$ where $\Delta^{\mathcal{I}} \neq \emptyset$ and $\cdot^{\mathcal{I}}$ assigns to each concept name A a set $A^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}}$ and to each base relation R a binary relation $R^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}} \times \Delta^{\mathcal{I}}$, subject to:

(S1) **One cluster.** For all $x, y \in \Delta^{\mathcal{I}}$, there exists $R \in N_R$ with $(x, y) \in R^{\mathcal{I}}$.
(S2) **JEPD.** For distinct $R, S \in N_R$, $R^{\mathcal{I}} \cap S^{\mathcal{I}} = \emptyset$.
(S3) **Composition.** For all $R, S \in N_R$: $R^{\mathcal{I}} \circ S^{\mathcal{I}} \ \subseteq \ \bigcup_{T \in \mathsf{comp}(R,S)} T^{\mathcal{I}}$, where comp is the RCC5 composition table (Table 1).
(S4) **Converse.** $\mathsf{PPI}^{\mathcal{I}} = (\mathsf{PP}^{\mathcal{I}})^{-1}$, $\mathsf{DR}^{\mathcal{I}} = (\mathsf{DR}^{\mathcal{I}})^{-1}$, $\mathsf{PO}^{\mathcal{I}} = (\mathsf{PO}^{\mathcal{I}})^{-1}$.
(S5) **Strong EQ.** $\mathsf{EQ}^{\mathcal{I}} = \{(x, x) \mid x \in \Delta^{\mathcal{I}}\}$.

Concept constructors are interpreted standardly: $(\exists R.C)^{\mathcal{I}} = \{x \mid \exists y.\, (x, y) \in R^{\mathcal{I}} \wedge y \in C^{\mathcal{I}}\}$, etc.

A concept C is *satisfiable* if there exists an interpretation $\mathcal{I}$ and $x \in \Delta^{\mathcal{I}}$ with $x \in C^{\mathcal{I}}$.

Remark 2.3 (Weak EQ semantics)**.** The variant with *weak* EQ (only $Id_{\Delta^{\mathcal{I}}} \subseteq \mathsf{EQ}^{\mathcal{I}}$, so EQ may also hold between distinct elements) reduces to the strong variant by adding the axiom $\top \sqsubseteq \forall \mathsf{EQ}.C \sqcap \forall \mathsf{EQ}^{-}.C$ for every relevant concept C and internalizing via the universal role $R_* = \mathsf{DR} \sqcup \mathsf{PO} \sqcup \mathsf{EQ} \sqcup \mathsf{PP} \sqcup \mathsf{PPI}$. See Wessel [12, Section 4.3]. We therefore work exclusively with strong EQ.

Lemma 2.4 (EQ normalization)**.** *Under strong EQ semantics, for every concept D:*

$$(\forall \mathsf{EQ}.D)^{\mathcal{I}} = D^{\mathcal{I}} \qquad \textit{and} \qquad (\exists \mathsf{EQ}.D)^{\mathcal{I}} = D^{\mathcal{I}}.$$

Therefore, every $\mathcal{ALCI}_{RCC5}$ concept C_0 can be preprocessed in linear time by replacing every subconcept $\forall \mathsf{EQ}.D$ with D and every $\exists \mathsf{EQ}.D$ with D, yielding an equisatisfiable concept C_0' whose closure $\mathsf{cl}(C_0')$ contains no EQ-modalities.

Proof. Since $\mathsf{EQ}^{\mathcal{I}} = \{(x, x) \mid x \in \Delta^{\mathcal{I}}\}$, the only EQ-neighbor of any element x is x itself. Thus $x \in (\forall \mathsf{EQ}.D)^{\mathcal{I}}$ iff $x \in D^{\mathcal{I}}$, and $x \in (\exists \mathsf{EQ}.D)^{\mathcal{I}}$ iff $x \in D^{\mathcal{I}}$. □

Convention. From this point on, we assume that C_0 has been *EQ-normalized*: $\mathsf{cl}(C_0)$ contains no concepts of the form $\exists \mathsf{EQ}.D$ or $\forall \mathsf{EQ}.D$. Pair-types, quasimodel conditions, and tableau rules therefore need only consider roles in $N_R \setminus \{\mathsf{EQ}\}$.

Remark 2.5 (Key structural properties)**.** In any $\mathcal{ALCI}_{RCC5}$ interpretation:

- PP is a strict partial order (irreflexive, asymmetric, transitive).
- There is no finite model property: the concept $(\exists \mathsf{PP}.\top) \sqcap (\forall \mathsf{PP}.\exists \mathsf{PP}.\top)$ is satisfiable only in infinite models.
- There is no tree model property: all models are complete graphs.
- The *universal role* $R_* = \mathsf{DR} \sqcup \mathsf{PO} \sqcup \mathsf{EQ} \sqcup \mathsf{PP} \sqcup \mathsf{PPI}$ is definable, enabling TBox internalization.
- Under strong EQ, a *difference role* $R_D = R_* \setminus \mathsf{EQ}$ is available, enabling limited nominal expressivity.

Table 1: The RCC5 composition table. Entry for row S, column R gives $\mathsf{comp}(R, S)$, i.e., the possible relations for (a, c) when $R(a, b)$ and $S(b, c)$. The symbol $*$ denotes N_R, i.e., all five relations are possible.

comp	$\mathsf{DR}(a,b)$	$\mathsf{PO}(a,b)$	$\mathsf{EQ}(a,b)$	$\mathsf{PPI}(a,b)$	$\mathsf{PP}(a,b)$
$\mathsf{DR}(b,c)$	$*$	DR, PO, PPI	DR	DR, PO, PPI	DR
$\mathsf{PO}(b,c)$	DR, PO, PP	$*$	PO	PO, PPI	DR, PO, PP
$\mathsf{EQ}(b,c)$	DR	PO	EQ	PPI	PP
$\mathsf{PP}(b,c)$	DR, PO, PP	PO, PP	PP	PO, EQ, PP, PPI	PP
$\mathsf{PPI}(b,c)$	DR	DR, PO, PPI	PPI	PPI	$*$

2.2 The Patchwork Property of RCC5

We recall the fundamental result from qualitative constraint reasoning that is the lynchpin of our decidability proof.

Definition 2.6 (Atomic constraint network)**.** An *atomic RCC5 constraint network* N over a set of variables V assigns to each ordered pair (v_i, v_j) with $v_i \neq v_j$ a single base relation $R_{ij} \in N_R \setminus \{\mathsf{EQ}\}$, and assigns EQ to each (v_i, v_i), respecting converse ($R_{ji} = \mathsf{inv}(R_{ij})$).

Definition 2.7 (Path consistency)**.** An atomic network N is *path-consistent* if for every triple of variables (v_i, v_j, v_k):

$$R_{ik} \in \mathsf{comp}(R_{ij}, R_{jk}).$$

Definition 2.8 (Consistency)**.** An atomic network N is *consistent* (or *realizable*) if there exists an $\mathcal{ALCI}_{RCC5}$ interpretation $\mathcal{I}$ and elements $d_v \in \Delta^{\mathcal{I}}$ for each $v \in V$ such that $(d_{v_i}, d_{v_j}) \in R_{ij}^{\mathcal{I}}$ for all i, j.

Theorem 2.9 (Renz & Nebel [10]; Renz [9])**.** *For RCC5 (and for RCC8), an atomic constraint network is consistent if and only if it is path-consistent.*

Corollary 2.10 (Patchwork / Amalgamation property)**.** *Let N_1 be a consistent atomic RCC5 network on variable set V_1 and N_2 a consistent atomic RCC5 network on V_2, with $V_1 \cap V_2 \neq \emptyset$ and N_1, N_2 agreeing on all pairs from $V_1 \cap V_2$. Then there exists a consistent atomic network N on $V_1 \cup V_2$ that extends both N_1 and N_2 (i.e., N agrees with N_1 on V_1-pairs and with N_2 on V_2-pairs).*

Proof. Any triple of variables in $V_1 \cup V_2$ that lies entirely in V_1 or entirely in V_2 is path-consistent (by the consistency of N_1 or N_2). For "cross-triples" involving variables from both $V_1 \setminus V_2$ and $V_2 \setminus V_1$, the missing edges (between these two sets) can be filled in greedily: for each such pair, choose any base relation that is path-consistent with all existing triples. Theorem 2.9 guarantees that a globally consistent completion exists. (See Renz [9] for the formal proof that RCC5 and RCC8 satisfy this amalgamation property.) □

Definition 2.11 (Disjunctive constraint network)**.** A *disjunctive RCC5 constraint network* $\mathcal{N}$ over a set of variables V assigns to each ordered pair (v_i, v_j) with $v_i \neq v_j$ a non-empty set $D_{ij} \subseteq N_R \setminus \{\mathsf{EQ}\}$ of *possible* base relations (and $\{\mathsf{EQ}\}$ to each self-pair), respecting converse: $D_{ji} = \{\mathsf{inv}(R) \mid R \in D_{ij}\}$.

The network is *path-consistent* if for every triple (v_i, v_j, v_k) and every $R_{ij} \in D_{ij}$, there exist $R_{ik} \in D_{ik}$ and $R_{jk} \in D_{jk}$ such that $R_{ik} \in \mathsf{comp}(R_{ij}, R_{jk})$.

The network is *consistent* (has a consistent *atomic refinement*) if one can choose a single $R_{ij} \in D_{ij}$ for each pair such that the resulting atomic network is consistent.

Theorem 2.12 (Full RCC5 tractability; Renz [9])**.** *The entire RCC5 algebra is a tractable subclass: a disjunctive RCC5 constraint network is consistent if and only if it is path-consistent.*

Theorem 2.12 is strictly stronger than the atomic patchwork property (Theorem 2.9). It asserts that for RCC5, one can go from *disjunctive* path-consistency to a global atomic solution—not merely from atomic path-consistency to realizability. This stronger result is the engine of our decidability proof: it allows us to solve constraint satisfaction problems where each edge has a *set* of possible base relations, as long as the network is path-consistent.

3 Monotonicity Along PP-Chains

Before developing the main proof techniques, we establish a structural property of $\mathcal{ALCI}_{RCC5}$ models that constrains the behavior of infinite PP-chains and explains why $\mathcal{ALCI}_{RCC5}$ is "tamer" than one might expect.

Lemma 3.1 (Upward monotonicity)**.** *Let $\mathcal{I}$ be an $\mathcal{ALCI}_{RCC5}$ interpretation containing a PP-chain $a_0, a_1, a_2, \ldots$ (i.e., $(a_i, a_{i+1}) \in \mathsf{PP}^{\mathcal{I}}$ for all i). For any $x \in \Delta^{\mathcal{I}}$ and any $i \geq 0$, the relation $R(x, a_{i+1})$ is constrained by $R(x, a_i)$ as follows:*

$R(x, a_i)$	*Possible values of* $R(x, a_{i+1})$
PP	PP
PO	PO, PP
DR	DR, PO, PP
PPI	PO, EQ, PP, PPI

Proof. Direct from the composition table: $R(x, a_{i+1}) \in \mathsf{comp}(R(x, a_i), \mathsf{PP})$. Reading column $\mathsf{PP}(a, b)$ in Table 1 for each row gives the claimed sets. □

Corollary 3.2. *Going upward along a PP-chain, the relation of an external node progresses monotonically through at most four phases. The "strength" ordering* DR < PO < PP *is respected (with* PPI *on a separate track that merges into* PO/PP*). Reversals* PP → PO*,* PO → DR*, or* PP → DR *are impossible.*

Lemma 3.3 (Downward persistence)**.** *Going* downward *along a PP-chain (i.e., from a_i to a_{i-1} via* PPI*), the relation $R(x, a_{i-1}) \in \mathsf{comp}(R(x, a_i), \mathsf{PPI})$ satisfies:*

$R(x, a_i)$	*Possible values of* $R(x, a_{i-1})$
DR	DR
PPI	PPI
PO	PO, PPI
PP	PO, EQ, PP, PPI

In particular, DR *and* PPI *are downward closed.*

Proof. By $R(x, a_{i-1}) \in \mathsf{comp}(R(x, a_i), \mathsf{PPI})$, reading column $\mathsf{PPI}(a, b)$ from Table 1. □

Remark 3.4. The monotonicity lemmas show that an infinite PP-chain can only "absorb" external nodes in a structured way: once a node becomes DR or PPI relative to a chain element, it stays that way for all elements below; once it becomes PP, it stays PP for all elements above. This regularity is what allows a finite quasimodel to faithfully represent the infinite model structure.

4 The Quasimodel Method

Throughout this section, fix a concept C_0 in negation normal form (NNF).

4.1 Fischer–Ladner Closure

Definition 4.1 (Closure)**.** The *closure* $\mathsf{cl}(C_0)$ is the smallest set containing $\mathsf{sub}(C_0)$ (all subconcepts of C_0) and, for every concept name $A \in N_C$ occurring in C_0, the concept $\neg A$. We identify $\neg\neg D$ with D. Note $|\mathsf{cl}(C_0)| \leq 2|C_0|$.

4.2 Types

Definition 4.2 (Type)**.** A *type* (for C_0) is a subset $\tau \subseteq \mathsf{cl}(C_0)$ satisfying:
(T1) If $D_1 \sqcap D_2 \in \tau$, then $D_1 \in \tau$ and $D_2 \in \tau$.
(T2) If $D_1 \sqcup D_2 \in \tau$, then $D_1 \in \tau$ or $D_2 \in \tau$.
(T3) For every concept name A: not both $A \in \tau$ and $\neg A \in \tau$.
We write $\mathsf{Tp}(C_0)$ for the set of all types. Clearly $|\mathsf{Tp}(C_0)| \leq 2^{|\mathsf{cl}(C_0)|} \leq 2^{2|C_0|}$.

Given an interpretation $\mathcal{I}$ and $x \in \Delta^{\mathcal{I}}$, the type of x is $\tau^{\mathcal{I}}(x) = \{D \in \mathsf{cl}(C_0) \mid x \in D^{\mathcal{I}}\}$.

4.3 Pair-Types (R-Compatibility)

Definition 4.3 (R-compatibility (strengthened))**.** Let $\tau_1, \tau_2 \in \mathsf{Tp}(C_0)$ and $R \in N_R \setminus \{\mathsf{EQ}\}$. The triple (τ_1, R, τ_2) is *R-compatible* if conditions (P1)–(P2) hold, together with the *chain propagation* conditions (P1′)–(P2′) when $R \in \{\mathsf{PP}, \mathsf{PPI}\}$:
(P1) For all $\forall R.D \in \mathsf{cl}(C_0)$: if $\forall R.D \in \tau_1$ then $D \in \tau_2$.
(P2) For all $\forall \mathsf{inv}(R).D \in \mathsf{cl}(C_0)$: if $\forall \mathsf{inv}(R).D \in \tau_2$ then $D \in \tau_1$.
(P1′) If $R = \mathsf{PP}$: for all $\forall \mathsf{PP}.D \in \mathsf{cl}(C_0)$, if $\forall \mathsf{PP}.D \in \tau_1$ then $\forall \mathsf{PP}.D \in \tau_2$.
If $R = \mathsf{PPI}$: for all $\forall \mathsf{PPI}.D \in \mathsf{cl}(C_0)$, if $\forall \mathsf{PPI}.D \in \tau_1$ then $\forall \mathsf{PPI}.D \in \tau_2$.
(P2′) If $R = \mathsf{PP}$: for all $\forall \mathsf{PPI}.D \in \mathsf{cl}(C_0)$, if $\forall \mathsf{PPI}.D \in \tau_2$ then $\forall \mathsf{PPI}.D \in \tau_1$.
If $R = \mathsf{PPI}$: for all $\forall \mathsf{PP}.D \in \mathsf{cl}(C_0)$, if $\forall \mathsf{PP}.D \in \tau_2$ then $\forall \mathsf{PP}.D \in \tau_1$.
We define:

$$\mathsf{Pair}(C_0) = \{(\tau_1, R, \tau_2) \mid \tau_1, \tau_2 \in \mathsf{Tp}(C_0),\ R \in N_R \setminus \{\mathsf{EQ}\},\ (\tau_1, R, \tau_2) \text{ is } R\text{-compatible}\}.$$

Note that since C_0 is EQ-normalized (Lemma 2.4), $\mathsf{cl}(C_0)$ contains no EQ-modalities and pair-types need only consider roles in $N_R \setminus \{\mathsf{EQ}\}$.

Remark 4.4. Conditions (P1′) and (P2′) are motivated by the *transitivity of* PP. If $(x, y) \in \mathsf{PP}^{\mathcal{I}}$ and $\forall \mathsf{PP}.D$ holds at x, then for any z with $(y, z) \in \mathsf{PP}^{\mathcal{I}}$, transitivity gives $(x, z) \in \mathsf{PP}^{\mathcal{I}}$, so D holds at z; hence $\forall \mathsf{PP}.D$ holds at y. Condition (P1′) captures this: *universal* PP*-restrictions propagate forward along* PP*-edges*. Condition (P2′) captures the symmetric backward propagation of $\forall \mathsf{PPI}$-restrictions.

These conditions are automatically satisfied by pairs extracted from any model, and they make the old condition (Q3) (PP-transitivity) *derivable* from pair-compatibility: if $(\tau_1, \mathsf{PP}, \tau_2)$ and $(\tau_2, \mathsf{PP}, \tau_3)$ are both PP-compatible, then by chaining (P1)/(P1′) one obtains (P1) for $(\tau_1, \mathsf{PP}, \tau_3)$, and by chaining (P2)/(P2′) one obtains (P2) for $(\tau_1, \mathsf{PP}, \tau_3)$; the chain propagation conditions themselves are inherited transitively. Hence $(\tau_1, \mathsf{PP}, \tau_3) \in \mathsf{Pair}(C_0)$ automatically.

Remark 4.5 (Converse closure)**.** $\mathsf{Pair}(C_0)$ is converse-closed: $(\tau_1, R, \tau_2) \in \mathsf{Pair}(C_0)$ implies $(\tau_2, \mathsf{inv}(R), \tau_1) \in \mathsf{Pair}(C_0)$. This is immediate because condition (P1) for the pair (τ_1, R, τ_2) is exactly condition (P2) for $(\tau_2, \mathsf{inv}(R), \tau_1)$, and the chain propagation conditions are similarly symmetric under inverses.

4.4 Triple Consistency

Definition 4.6 (Consistent triple)**.** A *triple* $(\tau_1, R_{12}, \tau_2, R_{13}, \tau_3, R_{23})$ is *consistent* if:

(Tr1) Each pair is compatible: (τ_1, R_{12}, τ_2), (τ_1, R_{13}, τ_3), (τ_2, R_{23}, τ_3) are all in $\mathsf{Pair}(C_0)$.

(Tr2) The composition table is respected:

$$\begin{aligned} R_{13} &\in \mathsf{comp}(R_{12}, R_{23}), \\ R_{23} &\in \mathsf{comp}(\mathsf{inv}(R_{12}), R_{13}), \\ R_{12} &\in \mathsf{comp}(R_{13}, \mathsf{inv}(R_{23})). \end{aligned}$$

We write $\mathsf{Trip}(C_0)$ for the set of all consistent triples.

4.5 Quasimodel

Definition 4.7 (Quasimodel (revised))**.** A *quasimodel* for C_0 is a triple $\mathcal{Q} = (T, P, \tau_0)$ where:

- $T \subseteq \mathsf{Tp}(C_0)$ is a non-empty set of types,
- $P \subseteq \{(\tau_1, R, \tau_2) \in \mathsf{Pair}(C_0) \mid \tau_1, \tau_2 \in T\}$ is a set of pair-types, each of which is R-compatible in the strengthened sense of Definition 4.3,
- P is *converse-closed*: $(\tau_1, R, \tau_2) \in P$ implies $(\tau_2, \mathsf{inv}(R), \tau_1) \in P$,
- $\tau_0 \in T$ is a distinguished type with $C_0 \in \tau_0$.

We write $\mathsf{DN}(\tau_1, \tau_2) = \{R \in N_R \setminus \{\mathsf{EQ}\} \mid (\tau_1, R, \tau_2) \in P\}$ for the *domain* of the edge (τ_1, τ_2). This defines a *type-level disjunctive network* (Definition 2.11) on T.

The triple (T, P, τ_0) must satisfy:

(Q1) Existential witness. For every $\tau \in T$ and every $\exists R.D \in \tau$, there exists $\tau' \in T$ with $D \in \tau'$ and $R \in \mathsf{DN}(\tau, \tau')$.

(Q2) Completeness. For every two *distinct* types $\tau_1, \tau_2 \in T$ with $\tau_1 \neq \tau_2$: $\mathsf{DN}(\tau_1, \tau_2) \neq \emptyset$.

(Q3) Algebraic closure. The type-level disjunctive network DN is *algebraically closed*: for every $\tau_1, \tau_2, \tau_3 \in T$ (not necessarily distinct) and every $R_{12} \in \mathsf{DN}(\tau_1, \tau_2)$, there exist $R_{13} \in \mathsf{DN}(\tau_1, \tau_3)$ and $R_{23} \in \mathsf{DN}(\tau_2, \tau_3)$ such that $R_{13} \in \mathsf{comp}(R_{12}, R_{23})$.

Remark 4.8 (Discussion of (Q3) variants)**.** Condition (Q3) is the standard *algebraic closure* (or weak path-consistency) condition from qualitative constraint reasoning: it asserts that every domain value participates in some composition-consistent extension to any third type. There is a *stronger* variant, sometimes called *strong path-consistency*:

> **(Q3s)** For every $R_{12} \in \mathsf{DN}(\tau_1, \tau_2)$ and every $R_{13} \in \mathsf{DN}(\tau_1, \tau_3)$, there exists $R_{23} \in \mathsf{DN}(\tau_2, \tau_3)$ with $R_{13} \in \mathsf{comp}(R_{12}, R_{23})$.

Condition (Q3s) fixes *two* edges and asks for a compatible third, while (Q3) fixes *one* and asks for two. The two conditions differ when distinct elements of the same type have different spatial relationships to a third type (the *representative mismatch* phenomenon).

For the **soundness** direction, (Q3) is readily extractable from any model (Theorem 5.1), whereas (Q3s) is not: two elements of the same type τ_1 may be related by different base relations R_{12} and R_{13} to elements of types τ_2 and τ_3 respectively, without any guarantee that the type-level domains can simultaneously support both.

For the **completeness** direction, (Q3s) gives a cleaner argument: it directly implies that the element-level constraint network in Claim 5.2 is (strongly) path-consistent, which by Theorem 2.12 gives global consistency. With the weaker (Q3), the element-level network is *not* automatically path-consistent, and the extension argument requires a more careful analysis (see Remark 5.3).

We use (Q3) in the quasimodel definition because it is *extractable*: every satisfiable concept has a quasimodel (soundness). The completeness direction (quasimodel $\Rightarrow$ model) uses the full

RCC5 tractability theorem and succeeds for quasimodels arising from models; the status for abstract quasimodels is discussed in Remark 5.3.

Remark 4.9 (Design rationale)**.** The revised definition addresses several issues in the earlier version:

1. **Strengthened pair-compatibility.** Conditions (P1′)–(P2′) ensure that universal PP/PPI restrictions propagate along PP-chains. This makes the old condition (Q3) (PP-transitivity) *derivable*: if $(\tau_1, \mathsf{PP}, \tau_2), (\tau_2, \mathsf{PP}, \tau_3) \in P$, then (P1′) gives $\forall \mathsf{PP}.D \in \tau_1 \Rightarrow \forall \mathsf{PP}.D \in \tau_2 \Rightarrow D \in \tau_3$, so $(\tau_1, \mathsf{PP}, \tau_3) \in \mathsf{Pair}(C_0)$.
2. **Converse closure.** By Remark 4.5, any subset of $\mathsf{Pair}(C_0)$ that is converse-closed keeps the converse pairs automatically consistent. This fixes the gap identified in the earlier Lemma 5.3.
3. **(Q3) replaces (Q1.a), (Q1.b).** The old definition required per-pair and per-triple embeddability witnesses. The new condition (Q3), together with the full RCC5 tractability theorem (Theorem 2.12), constrains the type-level network to be algebraically closed, which is needed for the extension step in the completeness construction.
4. **(Q2) weakened.** For a type realized by a single element in a one-element model, no non-EQ self-pair exists. We require (Q2) only for distinct types.

5 Main Theorem: Quasimodel Characterization

Theorem 5.1. *A concept C_0 is satisfiable in $\mathcal{ALCI}_{RCC5}$ if and only if there exists a quasimodel for C_0.*

The proof occupies the next two subsections.

5.1 Soundness: Model ⇒ Quasimodel

Proof of soundness. Let $\mathcal{I} = (\Delta^{\mathcal{I}}, \cdot^{\mathcal{I}})$ be an $\mathcal{ALCI}_{RCC5}$ interpretation with $x_0 \in C_0^{\mathcal{I}}$. Define:

$$T = \{\tau^{\mathcal{I}}(x) \mid x \in \Delta^{\mathcal{I}}\},$$
$$\tau_0 = \tau^{\mathcal{I}}(x_0).$$

We define the pair-type set P as the set of all *realized* triples:

$$P = \{(\tau_1, R, \tau_2) \mid \exists\, x_1, x_2 \in \Delta^{\mathcal{I}} \colon \tau^{\mathcal{I}}(x_i) = \tau_i,\ (x_1, x_2) \in R^{\mathcal{I}},\ R \in N_R \setminus \{\mathsf{EQ}\}\}.$$

$P \subseteq \mathsf{Pair}(C_0)$**.** If $(x_1, x_2) \in R^{\mathcal{I}}$ with $R \in N_R \setminus \{\mathsf{EQ}\}$, then $(\tau^{\mathcal{I}}(x_1), R, \tau^{\mathcal{I}}(x_2))$ is R-compatible. Condition (P1) holds because $\forall R.D \in \tau^{\mathcal{I}}(x_1)$ implies $D \in \tau^{\mathcal{I}}(x_2)$ by the semantics of $\forall$; (P2) holds symmetrically. For (P1′): if $R = \mathsf{PP}$ and $\forall \mathsf{PP}.D \in \tau^{\mathcal{I}}(x_1)$, then for any z with $(x_2, z) \in \mathsf{PP}^{\mathcal{I}}$, transitivity gives $(x_1, z) \in \mathsf{PP}^{\mathcal{I}}$, so $D \in \tau^{\mathcal{I}}(z)$, hence $\forall \mathsf{PP}.D \in \tau^{\mathcal{I}}(x_2)$; (P2′) holds by the symmetric argument for PPI.

Converse closure. If $(\tau_1, R, \tau_2) \in P$ is realized by (x_1, x_2), then $(\tau_2, \mathsf{inv}(R), \tau_1)$ is realized by (x_2, x_1), so P is converse-closed.

(Q1) Existential witness. Let $\tau = \tau^{\mathcal{I}}(x)$ and $\exists R.D \in \tau$. There exists y with $(x, y) \in R^{\mathcal{I}}$ and $y \in D^{\mathcal{I}}$. Set $\tau' = \tau^{\mathcal{I}}(y)$. Then $D \in \tau'$ and (τ, R, τ') is realized by (x, y), so $R \in \mathsf{DN}(\tau, \tau')$.

(Q2) Completeness. Let $\tau_1 \neq \tau_2 \in T$, realized by elements x, y. Since $x \neq y$ (distinct types), there is a unique $R \in N_R \setminus \{\mathsf{EQ}\}$ with $(x, y) \in R^{\mathcal{I}}$. This gives $(\tau_1, R, \tau_2) \in P$, so $\mathsf{DN}(\tau_1, \tau_2) \neq \emptyset$.

(Q3) Algebraic closure. Let $R_{12} \in \mathsf{DN}(\tau_1, \tau_2)$, realized by (x_1, x_2), and let $\tau_3 \in T$, realized by some x_3. If $\tau_2 \neq \tau_3$ then $x_2 \neq x_3$; if $\tau_2 = \tau_3$ there may be only one element of that type, in which case $x_2 = x_3$ and we handle this separately below.

Case $x_2 \neq x_3$: The model determines $R^*_{23} = R^{\mathcal{I}}(x_2, x_3) \in N_R \setminus \{\mathsf{EQ}\}$ and $R^*_{13} = R^{\mathcal{I}}(x_1, x_3)$. By model consistency, $R^*_{13} \in \mathsf{comp}(R_{12}, R^*_{23})$. Both $(\tau_1, R^*_{13}, \tau_3)$ and $(\tau_2, R^*_{23}, \tau_3)$ are realized, hence in P. Setting $R_{13} = R^*_{13}$ and $R_{23} = R^*_{23}$ satisfies (Q3).

Case $x_2 = x_3$ (unique element of type $\tau_2 = \tau_3$): Then $R^{\mathcal{I}}(x_2, x_3) = \mathsf{EQ}$. By $\mathsf{comp}(R_{12}, \mathsf{EQ}) = \{R_{12}\}$, we need $R_{13} = R_{12} \in \mathsf{DN}(\tau_1, \tau_3) = \mathsf{DN}(\tau_1, \tau_2)$. Since $R_{12} \in \mathsf{DN}(\tau_1, \tau_2) = \mathsf{DN}(\tau_1, \tau_3)$ already, and EQ satisfies the composition constraint trivially, (Q3) holds.

Erratum. The case $x_2 = x_3$ above is **incorrect**. Condition (Q3) requires $R_{23} \in \mathsf{DN}(\tau_2, \tau_3)$, but DN is defined over $N_R \setminus \{\mathsf{EQ}\}$ (Definition 4.7), so $\mathsf{EQ} \notin \mathsf{DN}(\tau_2, \tau_2)$. When $\tau_2 = \tau_3$ and there is a unique element of that type, $\mathsf{DN}(\tau_2, \tau_2) = \emptyset$ (no realized non-EQ pair), and (Q3) fails for any $R_{12} \in \mathsf{DN}(\tau_1, \tau_2)$. This means the soundness direction does *not* establish (Q3) as stated (with "not necessarily distinct" types) when a type has singleton multiplicity in the model. A correct statement would restrict (Q3) to *pairwise distinct* types, or add EQ as a distinguished element of $\mathsf{DN}(\tau, \tau)$ for all $\tau \in T$. □

5.2 Completeness: Quasimodel ⇒ Model

This is the non-trivial direction. The proof requires care: the constructed model is typically infinite (e.g., when C_0 forces infinite PP-chains), and we must ensure that edge assignments to newly created elements never trigger "dormant" universal restrictions in a way that introduces concepts not already present in the assigned types. The key insight is that *types are assigned once at element creation and never change*; pair-compatibility (Definition 4.3) pre-accounts for all universal restriction propagations.

Proof of completeness. Let $\mathcal{Q} = (T, P, \tau_0)$ be a quasimodel for C_0. We construct a model $\mathcal{I} = (\Delta^{\mathcal{I}}, \cdot^{\mathcal{I}})$ via a *Henkin-style step-by-step construction* and then prove that the resulting limit structure is a valid $\mathcal{ALCI}_{RCC5}$ model of C_0.

The construction

Stage 0. Create a single element e_1 with type $\tau(e_1) := \tau_0$. Set $\Delta_0 = \{e_1\}$. Define concept name interpretations: $A^{\mathcal{I}}$ will contain exactly those elements e with $A \in \tau(e)$.

Stage $n+1$. Let $\Delta_n = \{e_1, \ldots, e_m\}$ with types $\tau(e_i)$ and roles $R(e_i, e_j) \in N_R$ assigned for all $i \neq j$ (with $R(e_i, e_i) = \mathsf{EQ}$).

Enumerate all unsatisfied demands: pairs $(e_k, \exists R.D)$ where $\exists R.D \in \tau(e_k)$ but no element in Δ_n serves as a witness (i.e., no e_j with $R(e_k, e_j) = R$ and $D \in \tau(e_j)$). Consider the next unsatisfied demand $(e_k, \exists R.D)$.

By condition (Q1) of the quasimodel, there exists a witness type $\tau' \in T$ with $D \in \tau'$ and $(\tau(e_k), R, \tau') \in P$. Create a new element e_{m+1} with $\tau(e_{m+1}) := \tau'$. Set $R(e_k, e_{m+1}) := R$ and $R(e_{m+1}, e_k) := \mathsf{inv}(R)$.

The key step is assigning a role $R(e_i, e_{m+1})$ for every existing element $e_i \in \Delta_n$ with $i \neq k$.

Claim 5.2. *There exist roles $S_1, \ldots, S_m \in N_R \setminus \{\mathsf{EQ}\}$ (with $S_k = R$) such that:*

(a) $(\tau(e_i), S_i, \tau') \in P$ for all i.

(b) For all $i \neq j$: the triple (e_i, e_j, e_{m+1}) with roles $R(e_i, e_j)$, S_i, S_j satisfies $S_i \in \mathsf{comp}(R(e_i, e_j), S_j)$ and is in $\mathsf{Trip}(C_0)$.

Proof of Claim 5.2. We formulate the extension as a *disjunctive* constraint satisfaction problem. Define a disjunctive RCC5 constraint network N on $\{e_1, \ldots, e_m, e_{m+1}\}$ (Definition 2.11):

- For $i, j \leq m$: $N(e_i, e_j) = \{R(e_i, e_j)\}$ (singleton — the existing, consistent assignment).
- For $i \leq m$: $N(e_i, e_{m+1}) = D_i := \mathsf{DN}(\tau(e_i), \tau')$ — all roles forming a valid pair-type with τ'.
- D_k is further restricted to $\{R\}$ (the witness role from (Q1)).

We seek an atomic refinement: values $S_i \in D_i$ for each i such that $S_i \in \mathsf{comp}(R(e_i, e_j), S_j)$ for all $i \neq j$.

Step 1: Pairwise satisfiability. By condition (Q3) of the quasimodel, for every $i, j \leq m$ and the relation $R(e_i, e_j) \in \mathsf{DN}(\tau(e_i), \tau(e_j))$ (which holds by (I2)), there exist $S_i \in D_i$ and $S_j \in D_j$ with $S_i \in \mathsf{comp}(R(e_i, e_j), S_j)$. Hence every three-node sub-network $\{e_i, e_j, e_{m+1}\}$ of N is satisfiable.

Step 2: Algebraic closure enforcement. Apply the standard path-consistency enforcement algorithm to N: iteratively, for each triple (e_i, e_j, e_{m+1}), refine

$$D_i \leftarrow D_i \cap \big\{S_i \mid \exists\, S_j \in D_j \colon S_i \in \mathsf{comp}(R(e_i, e_j), S_j)\big\},$$

and symmetrically for D_j, until a fixed point is reached. (The singleton edges among $e_1, \ldots, e_m$ are never modified.)

At the fixed point, N is a path-consistent disjunctive RCC5 network (every remaining domain value has composition-compatible support through every third node). By the full RCC5 tractability theorem (Theorem 2.12), N is consistent: a simultaneous assignment $S_1, \ldots, S_m$ with $S_i \in D_i$ and $S_i \in \mathsf{comp}(R(e_i, e_j), S_j)$ for all $i \neq j$ exists.

Since each $S_i \in D_i \subseteq \mathsf{DN}(\tau(e_i), \tau')$, we have $(\tau(e_i), S_i, \tau') \in P$, giving condition (a). Composition consistency gives condition (b).

Non-emptiness of domains after enforcement. The argument above requires that no domain D_i becomes empty during enforcement. Condition (Q3) ensures that the *initial* domains support pairwise satisfiability (Step 1), so the enforcement has non-trivial starting material. However, (Q3) alone does not *formally* guarantee that the cascading refinements preserve non-empty domains: a value removed from D_j (due to one triple) could have been the sole support for a value in D_i (needed by another triple).

For quasimodels extracted from actual models (as in the soundness proof, Theorem 5.1), this cannot happen: the model provides a *realized* assignment that survives all enforcement steps (it is already globally consistent). More precisely, if the quasimodel (T, P, τ_0) was extracted from a model $\mathcal{I}$ with P consisting of realized pair-types, then at stage $n + 1$ of the construction, the model's own role assignments provide $S_i^* = R^{\mathcal{I}}(x_i, x_{m+1}) \in D_i$ for witnesses x_i of type $\tau(e_i)$. These values are globally composition-consistent (being model-realized) and hence survive enforcement.

See Remark 5.3 for further discussion of this subtlety. □

Remark 5.3 (The extension gap)**.** The proof of Claim 5.2 contains a subtlety that deserves careful attention. Condition (Q3) (algebraic closure of the type-level network DN) guarantees that every three-node sub-network of N is satisfiable (Step 1), but this does *not* automatically imply that the full disjunctive network N is path-consistent or consistent.

The gap arises because the element-level network N is a *partial refinement* of the type-level network: the edges among the base nodes $e_1, \ldots, e_m$ are fixed to specific atomic values $R(e_i, e_j)$, while the type-level domains $\mathsf{DN}(\tau(e_i), \tau(e_j))$ may contain additional values. Fixing an edge to a singleton can break the path-consistency that holds at the type level.

Concretely, consider types τ_1, τ_2, τ' with $\mathsf{DN}(\tau_1, \tau') = \{S\}$ (a singleton) and the parent edge $R = S'$ where $\mathsf{comp}(R(\tau_1, \tau_2), S') = \{S''\}$ with $S'' \neq S$. Then enforcement empties D_1, even though (Q3) holds at the type level.

The soundness direction of Theorem 5.1 is unaffected: the quasimodel is extracted from an actual model, and the model's realized role assignments provide a globally consistent solution that survives enforcement (the "non-emptiness" paragraph in the proof above). For the **completeness** direction, the extension step succeeds for quasimodels that arise from the soundness construction (where the model witnesses the consistency of each extension network). For *abstract* quasimodels satisfying (Q1)–(Q3) that do not arise from any model, the extension network may be inconsistent.

This means the characterization in Theorem 5.1 is established as an *if*: every satisfiable concept has a quasimodel (extracted from its model). The *only-if* direction (every quasimodel gives a model) holds for quasimodels extracted from models but not necessarily for arbitrary quasimodels.

Erratum: The "no false negatives" claim is wrong. The type elimination algorithm (Section 6) does *not* preserve model-extracted quasimodels. Although the model types form a valid quasimodel among themselves, the algorithm starts with *all* types in $\mathsf{Tp}(C_0)$, and (Q3) enforcement with the full type set kills DN entries that the model types need for (Q1) witnesses. Concretely, the concept $C_1 = \exists\mathsf{PO}.D \sqcap \exists\mathsf{DR}.(B \sqcap \forall\mathsf{PO}.\neg D) \sqcap \forall\mathsf{DR}.\neg D \sqcap \forall\mathsf{PP}.\neg D \sqcap \forall\mathsf{PPI}.\neg D$ is satisfiable (3-element model) but the algorithm rejects it (0 types survive). See Remark 6.2.

The stronger condition (Q3s) (Remark 4.8) would close this gap by ensuring element-level path-consistency directly, but at the cost of a soundness failure (not every satisfiable concept admits a quasimodel with (Q3s)). A fully tight characterization may require tracking element-level composition constraints beyond the type-level abstraction.

Set $R(e_i, e_{m+1}) := S_i$ and $R(e_{m+1}, e_i) := \mathsf{inv}(S_i)$ for all i. This extends the interpretation to $\Delta_{n+1} = \Delta_n \cup \{e_{m+1}\}$.

The limit model. Using a fair enumeration of demands (e.g., dovetailing), define:

$$\mathcal{I} = \bigcup_{n \geq 0} (\Delta_n, \cdot_n^{\mathcal{I}}).$$

The model invariant

We now prove that the construction maintains the following invariant at every stage, and that the limit model is a valid $\mathcal{ALCI}_{RCC5}$ interpretation satisfying C_0.

Lemma 5.4 (Construction invariant)**.** *At every stage $n \geq 0$, the partial model $(\Delta_n, \cdot_n^{\mathcal{I}})$ satisfies:*

(I1) ***Type membership.*** *For every $e \in \Delta_n$: $\tau(e) \in T$.*

(I2) ***Pair-type membership.*** *For every distinct $e_i, e_j \in \Delta_n$: $(\tau(e_i), R(e_i, e_j), \tau(e_j)) \in P$.*

(I3) ***Composition consistency.*** *For every triple of distinct elements $(e_i, e_j, e_k) \in \Delta_n$: $R(e_i, e_k) \in \mathsf{comp}(R(e_i, e_j), R(e_j, e_k))$.*

(I4) ***Type permanence.*** *The type $\tau(e)$ is assigned at the stage when e is created and is never subsequently modified.*

Proof. By induction on n.

Base case ($n = 0$): $\Delta_0 = \{e_1\}$ with $\tau(e_1) = \tau_0 \in T$. No pairs or triples exist, so (I2)–(I3) hold vacuously. (I4) holds trivially.

Inductive step ($n \to n+1$): Assume (I1)–(I4) hold for $\Delta_n = \{e_1, \ldots, e_m\}$. We add e_{m+1} with type $\tau' \in T$ ((I1) maintained) and role assignments $S_1, \ldots, S_m$ from Claim 5.2.

For (I2): Claim 5.2(a) gives $(\tau(e_i), S_i, \tau') \in P$ for all i. By converse and $P \subseteq \mathsf{Pair}(C_0)$, $(\tau', \mathsf{inv}(S_i), \tau(e_i)) \in P$. Existing pairs are unchanged.

For (I3): Claim 5.2(b) gives composition consistency for all new triples involving e_{m+1}. Existing triples are unchanged.

For (I4): the type of e_{m+1} is set to τ' and never revisited; no existing type is modified. □

Why dormant universal restrictions cannot fire

A central concern is whether the edge assignments at stage $n+1$ can activate universal restrictions in existing elements, adding concepts to their types and potentially causing clashes. The following lemma shows this cannot happen: pair-type membership (I2) *pre-accounts* for all universal restriction propagations.

Lemma 5.5 (No dormant activation)**.** *Let $e_i \in \Delta_n$ with $\forall S.D \in \tau(e_i)$, and let e_{m+1} be the element added at stage $n+1$ with type τ' and edge $R(e_i, e_{m+1}) = S_i$. Then:*

(a) *If $S_i = S$ (the restriction matches the new edge), then $D \in \tau'$. No concept needs to be "propagated" — it is already in the type.*

(b) *If $S_i \neq S$, the restriction $\forall S.D$ does not apply to this edge.*

Symmetrically, if $\forall \mathsf{inv}(S_i).D \in \tau'$, then $D \in \tau(e_i)$ — but $\tau(e_i)$ was assigned at an earlier stage and already contains D.

Proof. For part (a): by (I2), $(\tau(e_i), S_i, \tau') \in P \subseteq \mathsf{Pair}(C_0)$. Since $S_i = S$, the R-compatibility condition (P1) in Definition 4.3 requires: if $\forall S.D \in \tau(e_i)$, then $D \in \tau'$. So D is already present in τ' at the moment of creation.

For part (b): trivial, as $\forall S.D$ constrains only S-neighbors.

For the symmetric case: R-compatibility condition (P2) requires that if $\forall \mathsf{inv}(S_i).D \in \tau'$ and $(\tau(e_i), S_i, \tau') \in P$, then $D \in \tau(e_i)$. By (I4), $\tau(e_i)$ was fixed at creation and already contains D. □

Remark 5.6 (Composition cannot force unexpected propagation)**.** One might worry that composition constraints could force an edge value that triggers an unaccounted-for universal restriction. For instance: $R(e_a, e_b) = U$ and $R(e_b, e_{m+1}) = S_b$ might force $R(e_a, e_{m+1}) = S_a$ where S_a is the unique element of $\mathsf{comp}(U, S_b)$. If $\forall S_a.D \in \tau(e_a)$ and $D \notin \tau'$, we would have a problem.

This cannot happen. The role S_a was chosen from the domain $\{S \mid (\tau(e_a), S, \tau') \in P\}$ by Claim 5.2. The patchwork property guarantees that a global assignment exists within these domains while satisfying all composition constraints. Every role S_a in the solution satisfies $(\tau(e_a), S_a, \tau') \in P$, which by R-compatibility implies $D \in \tau'$ whenever $\forall S_a.D \in \tau(e_a)$. The composition constraints and the pair-compatibility constraints are solved *jointly*; the patchwork property ensures a simultaneous solution exists.

Concept truth in the limit model

We now prove that types faithfully determine concept truth.

Lemma 5.7 (Concept truth)**.** *In the limit model $\mathcal{I} = \bigcup_n (\Delta_n, \cdot_n^{\mathcal{I}})$, for every $e \in \Delta^{\mathcal{I}}$ and every $D \in \mathsf{cl}(C_0)$:*

$$D \in \tau(e) \implies e \in D^{\mathcal{I}}.$$

Proof. By structural induction on D.

Case $D = A$ (concept name): By construction, $A^{\mathcal{I}} = \{e \in \Delta^{\mathcal{I}} \mid A \in \tau(e)\}$. If $A \in \tau(e)$, then $e \in A^{\mathcal{I}}$.

Case $D = \neg A$: If $\neg A \in \tau(e)$, then $A \notin \tau(e)$ (by type condition (T3): a type cannot contain both A and $\neg A$). So $e \notin A^{\mathcal{I}}$, hence $e \in (\neg A)^{\mathcal{I}}$.

Case $D = D_1 \sqcap D_2$: If $D_1 \sqcap D_2 \in \tau(e)$, then $D_1 \in \tau(e)$ and $D_2 \in \tau(e)$ by type condition (T1). By induction, $e \in D_1^{\mathcal{I}}$ and $e \in D_2^{\mathcal{I}}$, so $e \in (D_1 \sqcap D_2)^{\mathcal{I}}$.

Case $D = D_1 \sqcup D_2$: If $D_1 \sqcup D_2 \in \tau(e)$, then $D_1 \in \tau(e)$ or $D_2 \in \tau(e)$ by type condition (T2). By induction, $e \in D_1^{\mathcal{I}}$ or $e \in D_2^{\mathcal{I}}$, so $e \in (D_1 \sqcup D_2)^{\mathcal{I}}$.

Case $D = \forall R.E$: Suppose $\forall R.E \in \tau(e)$ and $(e, e') \in R^{\mathcal{I}}$ (i.e., $R(e, e') = R$ for some $e' \in \Delta^{\mathcal{I}}$). By invariant (I2), $(\tau(e), R, \tau(e')) \in P$. By R-compatibility (P1), $E \in \tau(e')$. By induction hypothesis, $e' \in E^{\mathcal{I}}$.

Since this holds for *every* R-neighbor e', we have $e \in (\forall R.E)^{\mathcal{I}}$.

This is the step where dormant restrictions are handled. At the stage when e' is created, the edge $R(e, e')$ triggers the universal restriction $\forall R.E \in \tau(e)$. But Lemma 5.5 guarantees $E \in \tau(e')$ — the concept was placed in $\tau(e')$ at creation, not propagated after the fact. No concept is added to $\tau(e)$ or $\tau(e')$ at any later stage.

Case $D = \exists R.E$: Suppose $\exists R.E \in \tau(e)$. By the Henkin construction with fair enumeration, there eventually exists a stage n at which the demand $(e, \exists R.E)$ is processed: a new element e' is created with $\tau(e') = \tau'$ where $E \in \tau'$ and $R(e, e') = R$. By induction hypothesis, $e' \in E^{\mathcal{I}}$. Hence $e \in (\exists R.E)^{\mathcal{I}}$. □

The role of monotonicity along PP-chains

The model constructed above may contain infinite PP-chains (e.g., when $(\exists \mathsf{PP}.\top) \sqcap (\forall \mathsf{PP}.\exists \mathsf{PP}.\top) \in \tau_0$). The monotonicity lemmas (Section 3) explain why the finite quasimodel faithfully represents these infinite structures.

Consider an infinite PP-chain $e_0, e_1, e_2, \ldots$ in the limit model (with $R(e_i, e_{i+1}) = \mathsf{PP}$ for all i) and an external element x. By Lemma 3.1, the relation $R(x, e_i)$ follows a monotone progression:

$$\mathsf{DR} \rightarrow \mathsf{DR}, \mathsf{PO}, \mathsf{PP}; \quad \mathsf{PO} \rightarrow \mathsf{PO}, \mathsf{PP}; \quad \mathsf{PP} \rightarrow \mathsf{PP}; \quad \mathsf{PPI} \rightarrow \mathsf{PO}, \mathsf{EQ}, \mathsf{PP}, \mathsf{PPI}.$$

After at most 3 transitions (e.g., $\mathsf{DR} \rightarrow \mathsf{PO} \rightarrow \mathsf{PP}$), the relation *stabilizes*: once $R(x, e_k) = \mathsf{PP}$ for some k, $R(x, e_i) = \mathsf{PP}$ for all $i \geq k$. By Lemma 3.3, DR and PPI are downward persistent: once $R(x, e_k) = \mathsf{DR}$, $R(x, e_i) = \mathsf{DR}$ for all $i \leq k$.

This has three consequences for the Henkin construction:

1. **Bounded type variation.** Along the chain, only finitely many distinct pair-types $(\tau(x), S, \tau(e_i))$ arise (since the relation $S = R(x, e_i)$ takes at most $|N_R \setminus \{\mathsf{EQ}\}| = 4$ values and stabilizes). All such pair-types are in P.
2. **Uniform universal propagation.** Once $R(x, e_i)$ stabilizes to some role S, the same set of concepts $\{E \mid \forall S.E \in \tau(x)\}$ applies to all subsequent chain elements. No *new* concept enters any $\tau(e_i)$ from x's perspective beyond the stabilization point.
3. **Finite representability.** The types $\tau(e_i)$ along the chain are drawn from T (finitely many). The monotonicity ensures that the "relational profile" of the chain (the mapping from external elements to sequences of pair-types) eventually becomes periodic. The quasimodel captures all relevant type/pair-type/triple-type information, so the infinite chain is fully determined by its finite abstraction.

Without monotonicity, the relations along a PP-chain could potentially oscillate (e.g., $\mathsf{DR} \rightarrow \mathsf{PO} \rightarrow \mathsf{DR} \rightarrow \mathsf{PO} \rightarrow \cdots$), generating an unbounded variety of pair-type configurations. The monotonicity lemmas rule this out: the progression is one-directional, ensuring that the finite quasimodel is a faithful representation of the infinite model.

Verification of model properties

The limit model $\mathcal{I} = \bigcup_{n \geq 0} (\Delta_n, \cdot_n^{\mathcal{I}})$ is a valid $\mathcal{ALCI}_{RCC5}$ interpretation:

- *One cluster:* every pair of elements has a role assigned (by construction, each new element receives edges to all existing elements).
- *JEPD:* each pair has exactly one base relation (roles are assigned uniquely and permanently).
- *Composition:* every triple satisfies the composition table (invariant (I3), maintained at each stage).
- *Converse:* maintained by setting $R(e_{m+1}, e_i) = \mathsf{inv}(S_i)$.
- *Strong EQ:* $R(e, e) = \mathsf{EQ}$ for all e, and no distinct pair has EQ (since $S_i \in N_R \setminus \{\mathsf{EQ}\}$).
- *Concept truth:* by Lemma 5.7, $C_0 \in \tau_0 = \tau(e_1)$ implies $e_1 \in C_0^{\mathcal{I}}$.

Therefore $\mathcal{I} \models C_0$, and C_0 is satisfiable. □

6 Decidability and Complexity via Type Elimination

Theorem 6.1 (RETRACTED). ~~*Concept satisfiability in $\mathcal{ALCI}_{RCC5}$ is decidable in* ExpTime.~~

This theorem is wrong. The type-elimination algorithm described below rejects satisfiable concepts. See Remark 6.2 for the concrete counterexample and root cause analysis.

The original proof is retained (struck through) for reference:

Original proof (incorrect). By the soundness direction of Theorem 5.1, if C_0 is satisfiable then a quasimodel for C_0 exists. Equivalently, if no quasimodel exists then C_0 is unsatisfiable. We describe a type-elimination algorithm that decides quasimodel existence.

1. **Initialization.** Compute $\mathsf{Tp}(C_0)$, $\mathsf{Pair}(C_0)$, and $\mathsf{Trip}(C_0)$. These sets have sizes bounded by $2^{O(|C_0|)}$, $2^{O(|C_0|)}$, and $2^{O(|C_0|)}$ respectively. Set $T := \mathsf{Tp}(C_0)$ and $P := \{(\tau_1, R, \tau_2) \in \mathsf{Pair}(C_0) \mid \tau_1, \tau_2 \in T\}$.
2. **Algebraic closure enforcement.** Enforce condition (Q3) on the type-level disjunctive network DN: for each triple (τ_1, τ_2, τ_3) and each $R_{12} \in \mathsf{DN}(\tau_1, \tau_2)$, check whether there exist $R_{13} \in \mathsf{DN}(\tau_1, \tau_3)$ and $R_{23} \in \mathsf{DN}(\tau_2, \tau_3)$ with $R_{13} \in \mathsf{comp}(R_{12}, R_{23})$. If not, remove (τ_1, R_{12}, τ_2) from P and repeat until stable.
3. **Type elimination.** Iteratively remove from T any type τ that fails one of:
   - *Existential demand (Q1):* there exists $\exists R.D \in \tau$ for which no witness type $\tau' \in T$ with $D \in \tau'$ and $R \in \mathsf{DN}(\tau, \tau')$ can be found.
   - *Completeness (Q2):* there exists a distinct type $\tau' \in T$ with $\mathsf{DN}(\tau, \tau') = \emptyset$.

   When a type is removed, also remove all pair-types involving it, re-run algebraic closure enforcement (Step 2), then repeat.
4. **Termination.** Each iteration removes at least one type or pair-type. Since there are at most $2^{O(|C_0|)}$ types and $2^{O(|C_0|)}$ pair-types, termination occurs after at most $2^{O(|C_0|)}$ iterations.
5. **Acceptance.** Accept if and only if there remains a type τ_0 with $C_0 \in \tau_0$ such that τ_0 and the surviving types and pair-types form a quasimodel for C_0 (i.e., conditions (Q1)–(Q3) are satisfied).

Each iteration inspects at most $2^{O(|C_0|)}$ types, pairs, and triples. Total time: $2^{O(|C_0|)} \cdot 2^{O(|C_0|)} = 2^{O(|C_0|)}$, which is in ExpTime.

Correctness (INCORRECT). ~~If C_0 is satisfiable, the soundness proof (Section 5.1) produces a quasimodel (T^*, P^*, τ_0) whose types, pair-types, and algebraic closure property are all preserved by the elimination algorithm. Hence τ_0 survives, and the algorithm accepts.~~

The flaw: The quasimodel (T^*, P^*) extracted from a model is *not* preserved by the elimination algorithm. The algorithm starts with $T = \mathsf{Tp}(C_0) \supsetneq T^*$, and (Q3) enforcement with the full type set T removes pair-types that are valid within T^*. This is because (Q3) universally quantifies over *all* $\tau_3 \in T$: types in $T \setminus T^*$ may have $\mathsf{DN}(\tau_2, \tau_3) = \emptyset$, causing (Q3) to remove (τ_1, R_{12}, τ_2) even though the triple (τ_1, τ_2, τ_3) never arises in the model. The resulting (Q1) and (Q2) cascade eliminates all types, including τ_0. See Remark 6.2. □

Remark 6.2 (The type elimination bug)**.** The correctness argument for Theorem 6.1 fails because the greatest-fixpoint elimination does not preserve model-extracted quasimodels. The flaw is subtle: condition (Q3) is *anti-monotone* in the type set T (larger T makes (Q3) harder to satisfy), while (Q1) is anti-monotone in the opposite direction (smaller T makes (Q1) harder). The greatest-fixpoint approach assumes these conditions are compatible, but they are not.

Concrete counterexample. $C_1 = \exists\mathsf{PO}.D \sqcap \exists\mathsf{DR}.(B \sqcap \forall\mathsf{PO}.\neg D) \sqcap \forall\mathsf{DR}.\neg D \sqcap \forall\mathsf{PP}.\neg D \sqcap \forall\mathsf{PPI}.\neg D$ is satisfiable: the model $\{e_0, e_1, e_2\}$ with e_0 (root, $\neg B, \neg D$), e_1 (B-filler, $B, \neg D$), e_2 (D-element, $D, \neg B$), and edges $R(e_0, e_1) = \mathsf{DR}$, $R(e_0, e_2) = \mathsf{PO}$, $R(e_1, e_2) = \mathsf{DR}$ satisfies C_1 at e_0 (composition-consistent, verified computationally).

The three model-extracted types $\{\tau_0, \tau_1, \tau_2\}$ with compatible $\mathrm{DN} = \{(\tau_0, \tau_1) : \{\mathsf{DR}, \mathsf{PO}, \mathsf{PPI}\},\ (\tau_0, \tau_2) : \{\mathsf{PO}\},\ (\tau_1, \tau_2) : \{\mathsf{DR}\}\}$ form a valid quasimodel: (Q1) ✓, (Q2) ✓, (Q3) ✓ (pairwise-distinct triples).

However, $|\mathsf{Tp}(C_1)| = 128$. Among these, types with D and $\forall R.\neg D$ for all R have empty DN with all other D-types. The elimination cascade:

1. (Q3) with all 128 types prunes DN entries: $(\tau_0, \mathsf{PP}, \tau_1)$ is removed because there exists τ_3 (a D-type in $\mathsf{Tp}(C_1)$ with empty DN to τ_1).
2. (Q2) kills types whose DN to some surviving type becomes empty. This removes 78 types in the first iteration.
3. (Q1) kills types that lost their witnesses: 50 more types in the second iteration.
4. After 2 iterations: $T = \emptyset$, 0 types survive.

The fundamental issue. For standard $\mathcal{ALCI}$ (no spatial constraints), type elimination uses only (Q1). (Q1) is anti-monotone: the greatest fixpoint correctly finds the largest self-sustaining type set. For $\mathcal{ALCI}_{RCC5}$, (Q3) adds a condition that is anti-monotone in the *opposite* direction. These opposing forces make the greatest-fixpoint approach incorrect. A correct algorithm would need *subset search* (find $S \subseteq \mathsf{Tp}(C_0)$ where (Q1)–(Q3) hold internally), which is computationally harder.

Theorem 6.3 (Lower bound)**.** *Concept satisfiability in $\mathcal{ALCI}_{RCC5}$ is PSPACE-hard.*

Proof. Wessel [12, Section 6.3] establishes this by a reduction from QBF validity, using the PO-free fragment to enforce tree-shaped models. □

Corollary 6.4. *Concept satisfiability in $\mathcal{ALCI}_{RCC5}$ is between PSPACE and EXPTIME.*

Remark 6.5 (Closing the gap)**.** We conjecture that $\mathcal{ALCI}_{RCC5}$ is EXPTIME-complete. The interaction between PP-transitivity and the universal role (which enables TBox internalization) plausibly supports a reduction from the EXPTIME-hard *two-player corridor tiling problem*, but we have not established this.

7 A Tableau Decision Procedure

We now present an alternative decision procedure in the form of a *tableau calculus with blocking*. Unlike the type-elimination algorithm, which works globally on the space of all types and pair-types, the tableau constructs a model-like structure incrementally, expanding individual nodes. The key challenge is that $\mathcal{ALCI}_{RCC5}$ models are complete graphs, not trees: creating a new node requires assigning it a base relation to *every* existing node. The patchwork property (Theorem 2.9) is what makes this tractable.

7.1 Completion Graphs

Definition 7.1 (Completion graph)**.** A *completion graph* for C_0 is a tuple $\mathcal{G} = (V, \mathcal{L}, \mathcal{E}, \prec)$ where:

- V is a finite, non-empty set of *nodes*.
- $\mathcal{L}\colon V \to 2^{\mathsf{cl}(C_0)}$ is a *labelling function* assigning a set of concepts to each node.
- $\mathcal{E}\colon \{(x, y) \in V \times V \mid x \neq y\} \to N_R \setminus \{\mathsf{EQ}\}$ is an *edge labelling function* assigning a base relation to every ordered pair of distinct nodes, respecting converse: $\mathcal{E}(y, x) = \mathsf{inv}(\mathcal{E}(x, y))$.
- $\prec$ is a strict total order on V (the *creation order*).

The *initial completion graph* for C_0 consists of a single node x_0 with $\mathcal{L}(x_0) = \{C_0\}$, $V = \{x_0\}$, and $\mathcal{E}$ vacuously defined (no pairs of distinct nodes yet).

Remark 7.2. The edge labelling $\mathcal{E}$ is a *total* function on distinct pairs, reflecting the one-cluster and JEPD requirements. Whenever a new node y is added to V, the function $\mathcal{E}$ must be extended to assign a base relation $\mathcal{E}(z, y)$ for every existing node z. This is the principal difference from tree-based tableau calculi, where a new node has a single edge to its parent.

Definition 7.3 (Blocking)**.** Let $\mathcal{G} = (V, \mathcal{L}, \mathcal{E}, \prec)$ be a completion graph. A node $x \in V$ is *blocked* if there exists $y \prec x$ such that y is not blocked and $\mathcal{L}(x) = \mathcal{L}(y)$. We call y a *blocker* for x. A node is *active* if it is not blocked.

Remark 7.4. This is *equality anywhere-blocking*: the blocker y need not be a predecessor or neighbor in any structural sense—it need only precede x in the creation order and carry the same label. The simplicity of this blocking condition (compared to, e.g., the pairwise blocking of $\mathcal{ALCI}$ tree tableaux) is made possible by the patchwork property, which allows us to decouple the type-level structure from the specific relational neighborhood (see the soundness proof, Section 7.6).

7.2 Expansion Rules

The tableau calculus consists of four deterministic rules and two nondeterministic rules, applied to a completion graph $\mathcal{G}$. We write $\mathcal{L}(x)$ for the label and $\mathcal{E}(x, y)$ for the edge label.

Propositional rules (deterministic):

($\sqcap$) If $D_1 \sqcap D_2 \in \mathcal{L}(x)$ and $\{D_1, D_2\} \not\subseteq \mathcal{L}(x)$:
add D_1 and D_2 to $\mathcal{L}(x)$.

($\sqcup$) If $D_1 \sqcup D_2 \in \mathcal{L}(x)$ and $D_1 \notin \mathcal{L}(x)$ and $D_2 \notin \mathcal{L}(x)$:
nondeterministically add D_1 or D_2 to $\mathcal{L}(x)$.

Propagation rule (deterministic):

($\forall$) If $\forall R.D \in \mathcal{L}(x)$ and $\mathcal{E}(x, y) = R$ and $D \notin \mathcal{L}(y)$:
add D to $\mathcal{L}(y)$.

This rule also handles inverses: since $\forall R^{-}.D$ is equivalent to $\forall \mathsf{inv}(R).D$ in NNF, and $\mathcal{E}$ stores base relations, the rule fires whenever $\forall S.D \in \mathcal{L}(x)$ and $\mathcal{E}(x, y) = S$.

Generating rule (nondeterministic):

($\exists$) If $\exists R.D \in \mathcal{L}(x)$, x is active, and there is no $y \in V$ with $\mathcal{E}(x, y) = R$ and $D \in \mathcal{L}(y)$:

1. Create a fresh node y and add it to V as the $\prec$-greatest element.
2. Set $\mathcal{L}(y) := \{D\}$.
3. Set $\mathcal{E}(x, y) := R$ and $\mathcal{E}(y, x) := \mathsf{inv}(R)$.
4. For each $z \in V \setminus \{x, y\}$, *nondeterministically* choose $S_z \in N_R \setminus \{\mathsf{EQ}\}$ and set $\mathcal{E}(z, y) := S_z$, $\mathcal{E}(y, z) := \mathsf{inv}(S_z)$.

Remark 7.5. Step 4 is the hallmark of the complete-graph tableau: the new node y receives an edge to *every* existing node. The choice of S_z is nondeterministic, subject to the constraint-filtering condition below. The patchwork property guarantees that a valid choice exists whenever the concept is satisfiable (see Section 7.7).

7.3 Constraint Filtering

After the ($\exists$)-rule creates a new node y with edges to all existing nodes, we require:

(CF) Composition consistency. For every triple of distinct nodes (u, v, w) that includes y:

$$\mathcal{E}(u, w) \in \mathsf{comp}(\mathcal{E}(u, v),\ \mathcal{E}(v, w)).$$

If no assignment of the S_z values in step 4 of ($\exists$) satisfies (CF) for all such triples simultaneously, the current branch *closes* (clash by composition failure).

Remark 7.6. Checking (CF) amounts to solving an atomic RCC5 constraint network: the existing nodes have fixed pairwise relations, and the new node y has the relation to x fixed at R, with the remaining relations S_z as variables. By Theorem 2.9, a simultaneous satisfying assignment exists if and only if every triple involving y is individually satisfiable, i.e., the network is path-consistent. Path consistency can be checked (and enforced) in time $O(|V|^3)$.

7.4 Rule Application Strategy

To ensure that labels stabilize before blocking is evaluated:

1. Apply ($\sqcap$), ($\sqcup$), and ($\forall$) exhaustively to all nodes (in any order, resolving ($\sqcup$) nondeterministically).
2. Re-evaluate blocking: a node x is blocked if $\mathcal{L}(x) = \mathcal{L}(y)$ for some non-blocked $y \prec x$.
3. Apply one instance of ($\exists$) for an active node with an unsatisfied demand.
4. After adding the new node and its edges, apply ($\forall$) exhaustively (the new edges may trigger propagations both to and from the new node; these may cascade to other nodes).
5. Re-evaluate blocking (label changes from ($\forall$) may affect blocking status).
6. Repeat from step 1 until no rule applies or a clash is detected.

Definition 7.7 (Saturated completion graph)**.** A completion graph $\mathcal{G}$ is *saturated* if no expansion rule is applicable (i.e., all propositional rules have been applied, all ($\forall$)-propagations have been performed, and every active node's existential demands are witnessed by an appropriate neighbor).

Definition 7.8 (Clash-free completion graph)**.** A completion graph is *clash-free* if:
(C1) No node has $\{A, \neg A\} \subseteq \mathcal{L}(x)$ for any concept name A.
(C2) Composition consistency (CF) holds for all triples.
A completion graph that is both saturated and clash-free is called *open*.

The tableau *decides* satisfiability: C_0 is satisfiable iff there exists a sequence of nondeterministic choices producing an open completion graph.

7.5 Termination

Theorem 7.9 (Termination)**.** *Every sequence of rule applications terminates. The final completion graph has at most* $2^{O(|C_0|)}$ *nodes.*

Proof. Let $n = |\mathsf{cl}(C_0)| \leq 2|C_0|$ and $N = 2^n = |\mathsf{Tp}(C_0)|$.

Bound on active nodes. At any point, the active (non-blocked) nodes have pairwise distinct labels (by the definition of blocking). Each label is a subset of $\mathsf{cl}(C_0)$, so there are at most N active nodes.

Labels grow monotonically. The rules ($\sqcap$), ($\sqcup$), and ($\forall$) only *add* concepts to labels, never remove them. Therefore, each node's label changes at most n times (once per concept in $\mathsf{cl}(C_0)$).

Bound on node creation. The ($\exists$) rule fires only for active nodes with unsatisfied demands. Each active node x has at most n existential demands (concepts of the form $\exists R.D$ in $\mathsf{cl}(C_0)$). Each demand generates at most one new node (once a witness exists, the demand is satisfied and the rule no longer fires for that demand). With at most N active nodes, at most $N \cdot n$ nodes are created by the ($\exists$) rule. Each newly created node is either active (contributing to the at-most-N budget) or immediately blocked.

Bound on re-evaluations. When a label changes (via ($\forall$) propagation), blocking status may change, and a previously blocked node may become active, requiring expansion. However, each label change is permanent (labels only grow), and the total number of label changes across all nodes is at most $|V| \cdot n$. Each unblocking event is caused by a label change, so the total number of unblocking events is bounded. Each unblocking event may generate at most n new witnesses, but the total number of witnesses is bounded by the total number of active-node $\times$ demand pairs encountered during the process, which is at most $N \cdot n$ per "generation" of the label.

Overall bound. The total number of nodes is bounded by $N \cdot n \cdot n = 2^n \cdot n^2 = 2^{O(|C_0|)}$, and the total number of rule applications (label additions and node creations) is bounded by $|V| \cdot n = 2^{O(|C_0|)}$. Hence the procedure terminates in at most $2^{O(|C_0|)}$ steps. $\square$

7.6 Soundness: Open Branch ⇒ Satisfiable

Theorem 7.10 (Soundness of the tableau)**.** *If the tableau produces an open (saturated, clash-free) completion graph $\mathcal{G}$ for C_0, then C_0 is satisfiable in $\mathcal{ALCI}_{RCC5}$.*

Proof. We show that an open completion graph $\mathcal{G} = (V, \mathcal{L}, \mathcal{E}, \prec)$ encodes a valid quasimodel. The result then follows from Theorem 5.1.

Step 1: Quasimodel extraction. Define:

$$\begin{aligned} T &= \{\mathcal{L}(x) \mid x \in V,\ x \text{ is active}\}, \\ P &= \{(\mathcal{L}(x), \mathcal{E}(x,y), \mathcal{L}(y)) \mid x, y \in V,\ x \neq y\}, \\ \tau_0 &= \mathcal{L}(x_0). \end{aligned}$$

Step 2: Verification of (Q1). Let $\tau \in T$ and $\exists R.D \in \tau$. Then $\tau = \mathcal{L}(x)$ for some active node x. Since $\mathcal{G}$ is saturated, there exists $y \in V$ with $\mathcal{E}(x,y) = R$ and $D \in \mathcal{L}(y)$. Set $\tau' = \mathcal{L}(y)$. Then $D \in \tau'$ and $R = \mathcal{E}(x,y) \in \mathsf{DN}(\tau, \tau')$, verifying (Q1).

Step 3: Verification of (Q2). Since $\mathcal{E}$ is a total function on distinct pairs, for any distinct $\tau_1, \tau_2 \in T$ with active witnesses x_1, x_2, the edge $\mathcal{E}(x_1, x_2)$ provides a role in $\mathsf{DN}(\tau_1, \tau_2)$, so $\mathsf{DN}(\tau_1, \tau_2) \neq \emptyset$.

Step 4: Verification of (Q3). Let $\tau_1, \tau_2, \tau_3 \in T$ (not necessarily distinct) and $R_{12} \in \mathsf{DN}(\tau_1, \tau_2)$. Then there exist nodes $a, b \in V$ with $\mathcal{L}(a) = \tau_1$, $\mathcal{L}(b) = \tau_2$, and $\mathcal{E}(a,b) = R_{12}$. Since $\tau_3 \in T$, there exists a node $c \in V$ with $\mathcal{L}(c) = \tau_3$ (choosing any witness of τ_3; if $\tau_3 = \tau_1$ we may take $c = a$, and similarly for $\tau_3 = \tau_2$; for the self-pair case $\tau_1 = \tau_3$ with $c = a$, we have $R_{13} = \mathsf{EQ}$ and $\mathsf{comp}(R_{12}, R_{23}) \ni R_{12}$ trivially).

For the case of three distinct nodes a, b, c: the completion graph assigns $\mathcal{E}(a,c)$ and $\mathcal{E}(b,c)$. The composition-consistency condition (CF) on the triple (a, b, c) gives

$$\mathcal{E}(a,c) \in \mathsf{comp}(\mathcal{E}(a,b), \mathcal{E}(b,c)) = \mathsf{comp}(R_{12}, \mathcal{E}(b,c)).$$

Set $R_{13} = \mathcal{E}(a,c) \in \mathsf{DN}(\tau_1, \tau_3)$ and $R_{23} = \mathcal{E}(b,c) \in \mathsf{DN}(\tau_2, \tau_3)$. Then $R_{13} \in \mathsf{comp}(R_{12}, R_{23})$, which is exactly the algebraic closure condition (Q3).

Step 5: Conclusion. The extracted triple (T, P, τ_0) is a quasimodel for C_0. Moreover, this quasimodel arises from a model-like structure (the completion graph) in which all pair-types are realized by concrete node pairs. By the completeness argument of Theorem 5.1, and the non-emptiness guarantee for model-derived quasimodels (Remark 5.3), C_0 is satisfiable in $\mathcal{ALCI}_{RCC5}$. □

7.7 Completeness: Satisfiable ⇒ Open Branch

Theorem 7.11 (Completeness of the tableau)**.** *If C_0 is satisfiable in $\mathcal{ALCI}_{RCC5}$, then there exists a sequence of nondeterministic choices yielding an open completion graph for C_0.*

Proof. Let $\mathcal{I} = (\Delta^{\mathcal{I}}, \cdot^{\mathcal{I}})$ be a model with $d_0 \in C_0^{\mathcal{I}}$. We show that nondeterministic choices can be guided by $\mathcal{I}$ so that no clash arises and every demand is satisfied.

Guide function. We maintain a partial function $\pi \colon V \to \Delta^{\mathcal{I}}$ that maps each tableau node to a "corresponding" element in the model, with the invariant:

(I) $\mathcal{L}(x) \subseteq \tau^{\mathcal{I}}(\pi(x))$ for all $x \in V$.

(II) $\mathcal{E}(x,y) = R^{\mathcal{I}}(\pi(x), \pi(y))$ for all distinct $x, y \in V$ (where $R^{\mathcal{I}}(a,b)$ denotes the unique base relation holding between a and b in $\mathcal{I}$).

Initialization. Set $\pi(x_0) = d_0$. Then $\mathcal{L}(x_0) = \{C_0\}$ and $d_0 \in C_0^{\mathcal{I}}$, so invariant (I) holds.

Guiding the ($\sqcup$) rule. When $D_1 \sqcup D_2 \in \mathcal{L}(x)$, invariant (I) gives $D_1 \sqcup D_2 \in \tau^{\mathcal{I}}(\pi(x))$, so $\pi(x) \in (D_1 \sqcup D_2)^{\mathcal{I}}$, meaning $\pi(x) \in D_1^{\mathcal{I}}$ or $\pi(x) \in D_2^{\mathcal{I}}$. Choose the true disjunct. Invariant (I) is maintained.

Guiding the ($\exists$) rule. When $\exists R.D \in \mathcal{L}(x)$ and x is active, invariant (I) gives $\pi(x) \in (\exists R.D)^{\mathcal{I}}$, so there exists $d' \in \Delta^{\mathcal{I}}$ with $(\pi(x), d') \in R^{\mathcal{I}}$ and $d' \in D^{\mathcal{I}}$.

Create node y and set $\pi(y) = d'$. Then:

- $\mathcal{E}(x, y) = R = R^{\mathcal{I}}(\pi(x), \pi(y))$. Invariant (II) holds for the pair (x, y).
- For each existing node z, choose $S_z = R^{\mathcal{I}}(\pi(z), \pi(y))$ (the actual relation in the model). Set $\mathcal{E}(z, y) = S_z$. Invariant (II) holds for (z, y).
- Composition consistency (CF) holds because the model satisfies the composition table.

After ($\forall$)-propagation, every concept added to $\mathcal{L}(y)$ is of the form D' where $\forall S.D' \in \mathcal{L}(z)$ and $\mathcal{E}(z, y) = S$. By invariant (I), $\forall S.D' \in \tau^{\mathcal{I}}(\pi(z))$, so $(\pi(z), \pi(y)) \in S^{\mathcal{I}}$ implies $\pi(y) \in (D')^{\mathcal{I}}$, i.e., $D' \in \tau^{\mathcal{I}}(\pi(y))$. Similarly for propagation from y back to existing nodes. Hence invariant (I) is maintained for all affected labels.

Clash-freeness. Invariant (I) implies $\mathcal{L}(x) \subseteq \tau^{\mathcal{I}}(\pi(x))$, which is a consistent type, so $\{A, \neg A\} \not\subseteq \mathcal{L}(x)$. Invariant (II) implies $\mathcal{E}(x, y) = R^{\mathcal{I}}(\pi(x), \pi(y))$, and the model satisfies composition, so (CF) holds.

Demand satisfaction despite blocking. The guide function shows that all nondeterministic choices can be made consistently. The only remaining concern is that a blocked node x might have an unsatisfied demand.

Let x be blocked by y (so $\mathcal{L}(x) = \mathcal{L}(y)$ and y is active). Suppose $\exists R.D \in \mathcal{L}(x)$. Then $\exists R.D \in \mathcal{L}(y)$, and since y is active and the graph is saturated, there exists a node w with $\mathcal{E}(y, w) = R$ and $D \in \mathcal{L}(w)$.

The demand of x is $\exists R.D$. We need an R-neighbor of x containing D. Node w is an $\mathcal{E}(y, w)$-neighbor of y with role R, but $\mathcal{E}(x, w)$ might be a different role.

This does not cause a problem *for the tableau*: the tableau does not require blocked nodes' demands to be directly satisfied in the completion graph. The completion graph is not itself a model; it is a finite representation from which a model is constructed (via the quasimodel extraction and Henkin construction in Theorem 7.10). The quasimodel extracted in the soundness proof has the type $\mathcal{L}(x) = \mathcal{L}(y) = \tau \in T$, and the demand $\exists R.D$ of τ is witnessed by the type $\mathcal{L}(w) = \tau'$ via the pair-type $(\tau, R, \tau') \in P$. The Henkin construction produces a model satisfying all demands of all types, including those of blocked nodes.

The existence of the guided sequence of choices shows that the nondeterministic tableau has at least one execution producing a saturated, clash-free completion graph. □

7.8 Summary of the Tableau Procedure

Corollary 7.12 (NOT ESTABLISHED). ~~*The tableau calculus of Section 7.2, with blocking (Definition 7.3) and constraint filtering (Section 7.3), is a decision procedure for $\mathcal{ALCI}_{RCC5}$ concept satisfiability.*~~

This corollary is not established. The individual components:

- **Termination**: established (Theorem 7.9).
- **Completeness**: established (Theorem 7.11).
- **Soundness**: **not established**. Theorem 7.10 extracts a quasimodel from an open completion graph and invokes the Henkin construction (Theorem 5.1), which has the extension gap (Remark 5.3). The extension step requires solving a disjunctive constraint network at each stage; 1,911 counterexamples to pairwise-implies-global solvability were found computationally at $m = 3$. Without closing the extension gap, the tableau's soundness is unproven.

Remark 7.13 (Comparison with tree-based tableaux)**.** In standard $\mathcal{ALCI}$ tableaux, the completion structure is a forest, new nodes have a single edge to their parent, and blocking uses *pairwise* matching (comparing both a node and its parent with the blocker and its parent) to handle inverse roles. In the $\mathcal{ALCI}_{RCC5}$ tableau:

1. The completion structure is a *complete graph*, not a tree.
2. The ($\exists$)-rule creates edges to *all* existing nodes, not just one.
3. Composition consistency (CF) replaces the tree-structure invariant.
4. Blocking is *simpler*—just type equality, with no need for pairwise matching—because the patchwork property handles the relational context: the quasimodel extraction (Theorem 7.10) shows that type-level information suffices to guarantee model existence, regardless of the specific role assignments between nodes.
5. Soundness uses an indirect argument (quasimodel extraction + Henkin construction) rather than directly reading off the model from the completion graph.

Remark 7.14 (Practical optimizations)**.** An implementation of this tableau can benefit from:

- *Lazy role assignment*: Instead of choosing all $\mathcal{E}(z, y)$ at node creation, maintain a constraint network and only instantiate roles when needed for ($\forall$)-propagation.
- *Path consistency preprocessing*: Before node creation, run arc consistency on the constraint network to prune impossible role values.
- *Dependency-directed backtracking*: When a clash is detected, track which ($\sqcup$) and ($\exists$)-choices contributed, and backtrack to the most recent relevant choice.
- *Caching*: Since blocking is type-based, previously explored subtrees for a given type can be reused.

8 Extension to $\mathcal{ALCI}_{RCC8}$

The RCC8 calculus uses eight base relations: DC, EC, PO, EQ, TPP, TPPI, NTPP, NTPPI. The logic $\mathcal{ALCI}_{RCC8}$ is defined analogously to $\mathcal{ALCI}_{RCC5}$, using the RCC8 composition table. Both decision procedures extend to $\mathcal{ALCI}_{RCC8}$, but the transfer requires care because RCC8 and RCC5 differ in an important tractability property.

What transfers directly. The *atomic* patchwork property holds for RCC8: path-consistent atomic RCC8 networks are globally consistent (Theorem 2.9). Since the quasimodel and tableau constructions rely on this atomic property, the following components carry over unchanged:

- Types, pair-types, and the quasimodel conditions (Q1)–(Q3) are defined over the seven non-EQ RCC8 base relations $\{\mathsf{DC}, \mathsf{EC}, \mathsf{PO}, \mathsf{TPP}, \mathsf{TPPI}, \mathsf{NTPP}, \mathsf{NTPPI}\}$, using the RCC8 composition table.
- The soundness proof (satisfiable concept $\Rightarrow$ quasimodel) extracts types and pair-types from a model exactly as in Section 5.1. Condition (Q3) is verified from the model's composition closure.
- The type-elimination algorithm (Section 6) enforces algebraic closure and iteratively eliminates types; it runs in $2^{O(|C_0|)}$ time (the constant increases from 4 to 7 non-EQ relations, but the exponential bound is unchanged).
- The tableau calculus uses the RCC8 composition table for constraint filtering (CF). Blocking, termination, and the extraction of a quasimodel from an open completion graph proceed as in Section 7.

The tractability difference. For RCC5, the *full* algebra is tractable: disjunctive (non-atomic) path-consistent RCC5 networks are consistent (Theorem 2.12). For RCC8, this fails: consistency of disjunctive RCC8 constraint networks is NP-complete [10]. The maximal tractable subalgebras of RCC8 are proper subsets of the full algebra [9].

In the RCC5 proof, full tractability is invoked in Claim 5.2 (Step 3) to solve the disjunctive extension network that arises during the Henkin construction. For RCC8, this step is not available. However, the argument for *model-derived* quasimodels does not depend on full tractability: the model itself provides a globally consistent role assignment that survives the path-consistency enforcement (see the non-emptiness paragraph in the proof of Claim 5.2). Consequently:

- The soundness direction of the characterization theorem (satisfiable $\Rightarrow$ quasimodel) holds for RCC8.
- The completeness direction (quasimodel $\Rightarrow$ model via Henkin) holds for quasimodels extracted from models, by the same argument as for RCC5.
- The extension gap (Remark 5.3) applies equally: for *abstract* quasimodels not arising from models, the Henkin construction may fail. Unlike RCC5, where full tractability provides an alternative route to solving the extension network, no such fallback is available for RCC8.
- **The type-elimination algorithm is unsound for RCC8 as well**, since the same anti-monotonicity of (Q3) applies (see Remark 6.2). The RCC8 algorithm inherits the cascade-elimination bug from the RCC5 version.
- The tableau's soundness proof extracts a quasimodel from an open completion graph—a structure where all pair-types are realized by concrete node pairs and composition consistency (CF) holds—and invokes the Henkin construction for model-derived quasimodels.

Theorem 8.1 (**RETRACTED**). ~~*Concept satisfiability in $\mathcal{ALCI}_{RCC8}$ is decidable in ExpTime.*~~

This theorem is wrong. It depends on the type-elimination algorithm (Theorem 6.1), which is unsound. The same anti-monotonicity of (Q3) that invalidates the RCC5 algorithm also invalidates the RCC8 version.

Original proof (incorrect). By the analysis above: the type-elimination algorithm is a correct ExpTime decision procedure for $\mathcal{ALCI}_{RCC8}$. The soundness direction uses the RCC8 composition table and is independent of tractability. The algorithm's acceptance criterion (survival of τ_0 through elimination) ensures no false negatives. The ExpTime time bound follows from the same analysis as for $\mathcal{ALCI}_{RCC5}$ (Section 6), with the constant factor increasing from 4 to 7 non-EQ base relations. □

Theorem 8.2 (Wessel [12]). *Concept satisfiability in $\mathcal{ALCI}_{RCC8}$ is ExpTime-hard.*

Proof. By reduction from the two-player corridor tiling game [12, Section 6.4]. □

Corollary 8.3 (**RETRACTED**). ~~*Concept satisfiability in $\mathcal{ALCI}_{RCC8}$ is ExpTime-complete.*~~

Retracted: depends on Theorem 8.1, which is wrong. The ExpTime lower bound (Theorem 8.2) remains valid. Decidability of $\mathcal{ALCI}_{RCC8}$ is **open**.

9 Related Work

Wessel's $\mathcal{ALCRA}_{SG}$ (2000/2001). The $\mathcal{ALCRA}_{SG}$ framework [11] handles role boxes that are *admissible*: deterministic (each composition yields a single relation), functional, complete, and associative. For such role boxes, the logic reduces to $\mathcal{ALC}$ with general TBoxes, yielding ExpTime-completeness. The RCC5 and RCC8 role boxes are non-deterministic (many composition entries yield sets of relations) and hence fall outside $\mathcal{ALCRA}_{SG}$. Our quasimodel and tableau methods handle this non-determinism by working with all possible completions simultaneously, guided by the patchwork property.

Lutz & Wolter (2006): Modal logics of topological relations. Lutz and Wolter [7] study modal logics with RCC8 modalities interpreted over *actual topological spaces*. In their setting, models must correspond to genuine spatial configurations (e.g., in $\mathbb{R}^n$), which imposes additional constraints beyond the composition table. This leads to undecidability in many cases. By contrast, $\mathcal{ALCI}_{RCC}$ uses *abstract* semantics: any interpretation satisfying the composition table constraints is admissible. The abstract semantics are more permissive, and this permissiveness is exactly what makes our decidability proof work.

Lutz & Miličić (2007): ω-admissible concrete domains. Lutz and Miličić [6] prove that $\mathcal{ALC}$ extended with ω-admissible concrete domains is decidable. RCC5 and RCC8 are ω-admissible, so $\mathcal{ALC}$(RCC5) and $\mathcal{ALC}$(RCC8) are decidable. However, the concrete-domain approach is fundamentally different: spatial predicates constrain *concrete features* of domain elements via functional roles, and one cannot quantify *over* spatial relations. In particular, $\forall\mathsf{PP}.C$ and $\exists\mathsf{DR}.D$ are not expressible in $\mathcal{ALC}$(RCC5). The $\mathcal{ALCI}_{RCC}$ logics are strictly more expressive for spatial reasoning because spatial relations serve directly as roles.

Borgwardt, De Bortoli & Koopmann (2024). Borgwardt et al. [2] sharpen the complexity of reasoning in $\mathcal{ALC}$ with ω-admissible concrete domains. Their results apply to the concrete-domain setting and do not directly bear on $\mathcal{ALCI}_{RCC}$, but confirm that the patchwork/amalgamation property of RCC calculi is a powerful tool for decidability in description logics with spatial features.

Baader & Rydval (2020): Concrete domains revisited. Baader and Rydval [3] strengthened undecidability results for description logics with concrete domains and general concept inclusions, and generalized the ω-admissibility conditions. Their results refine the boundary between decidable and undecidable concrete-domain extensions, but remain within the concrete-domain paradigm (functional roles to concrete values) and do not address composition-based role boxes.

Demri & Gu (2026): Constraint automata with inverse roles. Demri and Gu [4] extend the automata-based approach to description logics with concrete domains to handle *inverse roles*, functional role names, and constraint assertions, establishing ExpTime membership. This is the closest published result to $\mathcal{ALCI}_{RCC}$, as it combines inverse roles with RCC-like spatial constraints. However, it still operates within the concrete-domain formalism: spatial predicates constrain concrete features via functional roles, and one cannot quantify over spatial relations as roles. In $\mathcal{ALCI}_{RCC5}$, concepts like $\forall\mathsf{PP}.C$ and $\exists\mathsf{DR}.D$—which use spatial relations directly as roles—remain inexpressible in the concrete-domain setting. The two formalisms are thus complementary: the concrete-domain approach is well-suited for reasoning about spatial *attributes* of domain elements, while the $\mathcal{ALCI}_{RCC}$ approach captures direct spatial *relationships* between elements.

Constraint satisfaction and qualitative reasoning. The patchwork property (Theorem 2.9) was established by Renz and Nebel [10] and Renz [9] in the context of qualitative spatial reasoning and constraint satisfaction. Our contribution is to bridge the gap between this CSP result and the description logic setting: we show that the patchwork property, when combined with the quasimodel method or a tableau calculus, is precisely the tool needed to decide satisfiability in $\mathcal{ALCI}_{RCC}$ logics.

10 Discussion and Open Questions

1. **Decidability of $\mathcal{ALCI}_{RCC5}$ and $\mathcal{ALCI}_{RCC8}$.** Both problems remain **open**. The PSPACE lower bound for $\mathcal{ALCI}_{RCC5}$ [12] and the EXPTIME lower bound for $\mathcal{ALCI}_{RCC8}$ [12] remain valid. An EXPTIME upper bound was claimed in this paper but depends on the type-elimination algorithm (Theorem 6.1), which is unsound (Remark 6.2). Finding a correct decision procedure is the primary open problem.

2. **General characterization.** Which composition tables (beyond RCC5 and RCC8) yield decidable $\mathcal{ALCI}_{RCC}$-style logics? Our proof shows that the patchwork property is sufficient. It would be valuable to characterize exactly which qualitative calculi have this property and whether it is also necessary for decidability.

3. **Adding number restrictions.** Unqualified number restrictions make $\mathcal{ALCRA}_{SG}$ undecidable [11]. We expect the same for $\mathcal{ALCI}_{RCC5}$: number restrictions would allow enforcing specific branching patterns that, combined with concept distinctions, may enable grid encodings.

4. **Finite model reasoning.** Satisfiability restricted to *finite* models (ruling out infinite PP-chains) may be decidable in a lower complexity class, since the need for infinite witnesses—and hence the full power of the Henkin construction—disappears.

5. **Hybrid extensions.** Wessel [12] considers hybrid logic extensions with nominals and the @-operator. Whether our decidability results extend to these enrichments is an open question; the interaction between nominals and the complete-graph semantics may introduce new complications.

Acknowledgments

This research was prompted by Michael Wessel, who introduced the $\mathcal{ALCI}_{RCC}$ family during his doctoral work at the University of Hamburg under the DFG project "Description Logics and Spatial Reasoning" (grant NE 279/8-1) and left the decidability of $\mathcal{ALCI}_{RCC5}$ and $\mathcal{ALCI}_{RCC8}$ as open problems. The present work directly builds on his foundational contributions [12, 11].

This revised version corrects several issues identified in a technical review of the original manuscript: the treatment of EQ in the syntax and pair-types (Section **??**), the quasimodel conditions and their extractability from models (Section 4), the distinction between algebraic closure and full path-consistency (Remark 4.8), the Henkin construction and its reliance on full RCC5 tractability (Claim 5.2 and Remark 5.3), and the RCC8 extension (Section 8). We are grateful for the detailed and constructive review.

References

[1] F. Baader, D. Calvanese, D. McGuinness, D. Nardi, and P. Patel-Schneider, editors. *The Description Logic Handbook: Theory, Implementation, and Applications.* Cambridge University Press, 2003.

[2] S. Borgwardt, F. De Bortoli, and P. Koopmann. The precise complexity of reasoning in $\mathcal{ALC}$ with ω-admissible concrete domains. In *Proc. KR*, 2024.

[3] F. Baader and M. Rydval. Description logics with concrete domains and general concept inclusions revisited. In *Proc. IJCAR*, volume 12166 of LNCS, pages 413–431. Springer, 2020.

[4] S. Demri and T. Gu. Robustness of constraint automata for description logics with concrete domains. In *Proc. CSL*, volume 363 of LIPIcs, 2026.

[5] E. Grädel and M. Otto. On logics with two variables. *Theoretical Computer Science*, 224:73–113, 1999.

[6] C. Lutz and M. Miličić. A tableau algorithm for description logics with concrete domains and general TBoxes. *Journal of Automated Reasoning*, 38:227–259, 2007.

[7] C. Lutz and F. Wolter. Modal logics of topological relations. *Logical Methods in Computer Science*, 2(2), 2006.

[8] D. Randell, Z. Cui, and A. Cohn. A spatial logic based on regions and connection. In *Proc. KR*, pages 165–176, 1992.

[9] J. Renz. Maximal tractable fragments of the Region Connection Calculus: A complete analysis. In *Proc. IJCAI*, pages 448–455, 1999.

[10] J. Renz and B. Nebel. On the complexity of qualitative spatial reasoning: A maximal tractable fragment of the Region Connection Calculus. *Artificial Intelligence*, 108(1-2):69–123, 1999.

[11] M. Wessel. Decidable and undecidable extensions of $\mathcal{ALC}$ with composition-based role inclusion axioms. Technical Report FBI-HH-M-301/01, University of Hamburg, 2000.

[12] M. Wessel. Qualitative spatial reasoning with the $\mathcal{ALCI}_{RCC}$ family—First results and unanswered questions. Technical Report FBI-HH-M-324/03, University of Hamburg, 2002/2003.